

Mastering Lambdas: Java Programming in a
Multicore World



精通lambda表达式： **Java**多核编程

使用lambda表达式和流的最佳实践

[美] Maurice Naftalin 著
张 龙 译

清华大学出版社

精通 lambda 表达式： Java 多核编程

[美] Maurice Naftalin 著

张 龙 译

清华大学出版社

北 京

Maurice Naftalin

Mastering Lambdas: Java Programming in a Multicore World

EISBN: 978-0-07-182962-5

Copyright © 2015 by Maurice Naftalin.

All Rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means, electronic or mechanical, including without limitation photocopying, recording, taping, or any database, information or retrieval system, without the prior written permission of the publisher.

This authorized Chinese translation edition is jointly published by McGraw-Hill Education and Tsinghua University Press Limited. This edition is authorized for sale in the People's Republic of China only, excluding Hong Kong, Macao SAR and Taiwan.

Copyright © 2015 by McGraw-Hill Education and Tsinghua University Press Limited.

版权所有。未经出版人事先书面许可，对本出版物的任何部分不得以任何方式或途径复制或传播，包括但不限于复印、录制、录音，或通过任何数据库、信息或可检索的系统。

本授权中文简体字翻译版由麦格劳-希尔(亚洲)教育出版公司和清华大学出版社有限公司合作出版。

此版本经授权仅限在中华人民共和国境内(不包括中国香港、澳门特别行政区和中国台湾地区)销售发行。

版权©2015 由麦格劳-希尔(亚洲)教育出版公司与清华大学出版社有限公司所有。

北京市版权局著作权合同登记号 图字：01-2015-1599

本书封面贴有 McGraw-Hill Education 公司防伪标签，无标签者不得销售。

版权所有，侵权必究。侵权举报电话：010-62782989 13701121933

图书在版编目(CIP)数据

精通 lambda 表达式: Java 多核编程 / (美) 那夫特林(Naftalin, M.) 著; 张龙 译. —北京: 清华大学出版社, 2015

书名原文: Mastering Lambdas: Java Programming in a Multicore World

ISBN 978-7-302-40553-5

I. ①精… II. ①那… ②张… III. JAVA 语言—程序设计 IV. ①TP312

中国版本图书馆 CIP 数据核字(2015)第 137442 号

责任编辑: 王 军 于 平

装帧设计: 孔祥峰

责任校对: 曹 阳

责任印制:

出版发行: 清华大学出版社

网 址: <http://www.tup.com.cn>, <http://www.wqbook.com>

地 址: 北京清华大学学研大厦 A 座 邮 编: 100084

社 总 机: 010-62770175 邮 购: 010-62786544

投稿与读者服务: 010-62776969, c-service@tup.tsinghua.edu.cn

质 量 反 馈: 010-62772015, zhiliang@tup.tsinghua.edu.cn

装 订 者:

经 销: 全国新华书店

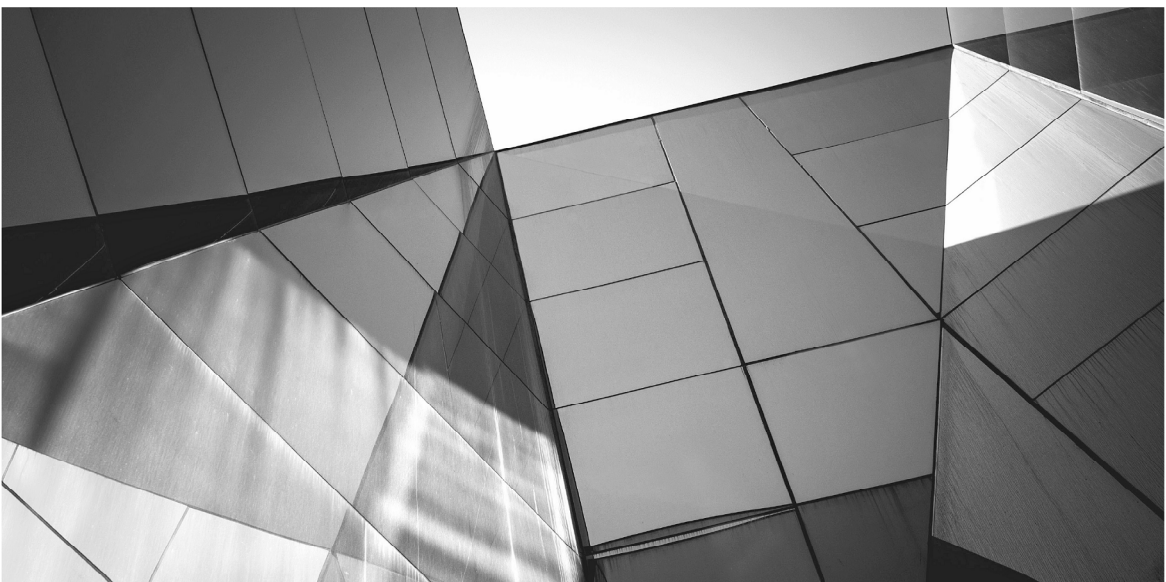
开 本: 148mm×210mm 印 张: 7.5 字 数: 181 千字

版 次: 2015 年 8 月第 1 版 印 次: 2015 年 8 月第 1 次印刷

印 数: 1~2000

定 价: 39.00 元

产品编号:



译者序

2014年3月,万众期待的Java 8终于发布了。Java 8可谓Java语言历史上变化最大的一个版本,这体现在语言、库与虚拟机的增强上。Java语言现在已经渗透到软件行业的各个领域,从大数据领域的Hadoop到企业级开发的Spring框架,再到移动领域的Android,无处不显现Java的身影。作为一门有着近20年历史的语言,Java影响了整整一代程序员。随着时代的变迁,动态语言、函数式编程越来越受到广大开发人员的关注,从这个角度来看,Java似乎有些臃肿和笨重。但强大的JVM赋予了Java新的生命力,有越来越多的语言能够运行在Java虚拟机上并与Java交互,如Groovy、Scala、Clojure等,这些语言反过来又促进了Java的

持续发展。

Java 8 引入了诸多令人心潮澎湃的新特性，如 lambda 表达式、Stream API、并发库的增强、默认方法、函数式接口、方法与构造方法引用、新的日期与时间 API、注解增强等，这些新特性促进了 Java 的持续发展。在 Java 8 的诸多特性当中，lambda 表达式是最为人所关注的。借助于 lambda 表达式，可以编写出性能更好、可读性更强、可维护性更好的代码，本书就是一本专门介绍 Java lambda 表达式的著作。

本书共分为 7 章，按照由浅入深、循序渐进的方式对 Java lambda 表达式进行了深入的剖析与介绍。首先从 Java 8 新特性开始，对 lambda 表达式作一简要说明；接下来介绍 lambda 表达式的基础，让读者对 lambda 表达式有个感性的认识；然后对流与管道进行深入的讲解，同时剖析起始流、终止流，并对流的性能做了详尽的阐述；最后，介绍默认方法的含义及其对于 API 演化的重要意义。本书不仅介绍 Java lambda 表达式的基础知识，还深入讲解了构成 lambda 表达式的重要组件，并对其适用场景进行了广泛的说明。

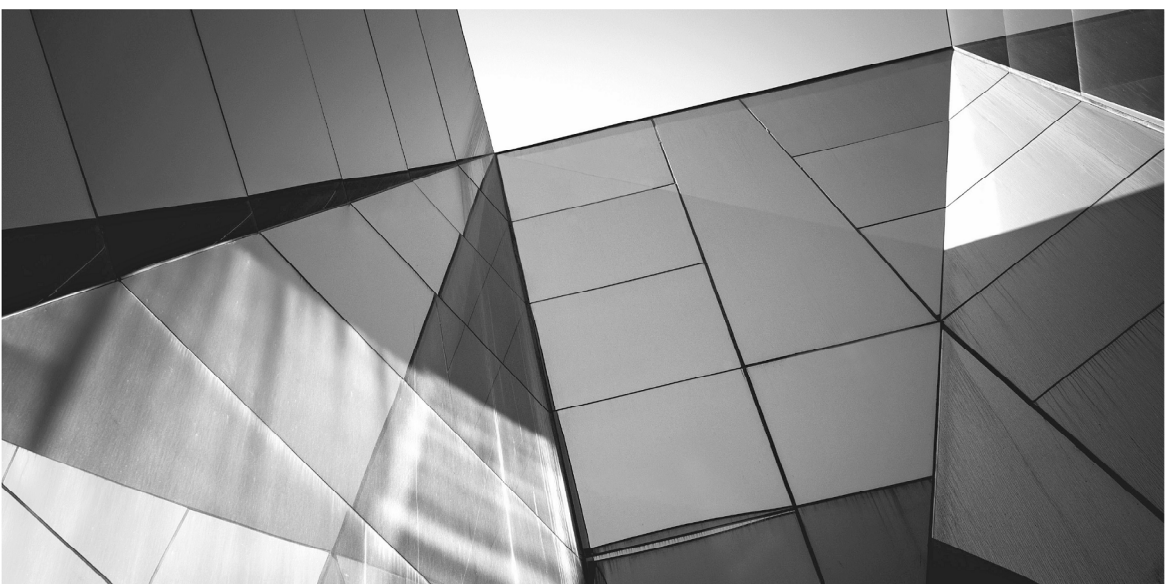
虽然现在很多公司开发所用的 JDK 尚未升级到最新的 JDK 8，但学习并实践 Java 8 的新特性是每一个有追求的 Java 开发者都应该做的事情，因为这是未来 Java 的发展方向。熟练掌握并灵活运用 lambda 表达式可以帮助你编写出更加优雅的代码，只需要寥寥数行代码即可解决之前十几行、甚至几十行代码才能完成的工作。但不得不提的是，要想熟练掌握 lambda 表达式并不是一件简单轻松的工作，因为其背后有很多重要理论需要学习者充分理解，而本书则可以帮助你更快、更好地掌握 lambda 表达式，因为书中所介绍的内容不仅会让你知其然，更会让你知其所以然。学习完本书后，我相信每一位读者都能够流畅地使用 lambda 表达式开始全

新 Java 代码的编写。

翻译技术图书是艰苦的劳动，不仅有体力的付出，更有大量脑力的付出。在此，请容许我感谢清华大学出版社的编辑，非常感谢他们在图书翻译过程中对我的包容与鼓励；其次，我要感谢我的父母，没有你们的精心抚育与教诲就不会有今天的我；再次，我将深深的感激之情送给我的妻子张明辉，谢谢你在图书翻译过程中给予我持续不断的支持与理解，本书的顺利出版有你一半的功劳；最后，将本书送给我亲爱的孩子张梓轩小朋友，愿你未来能够开开心心，茁壮成长。

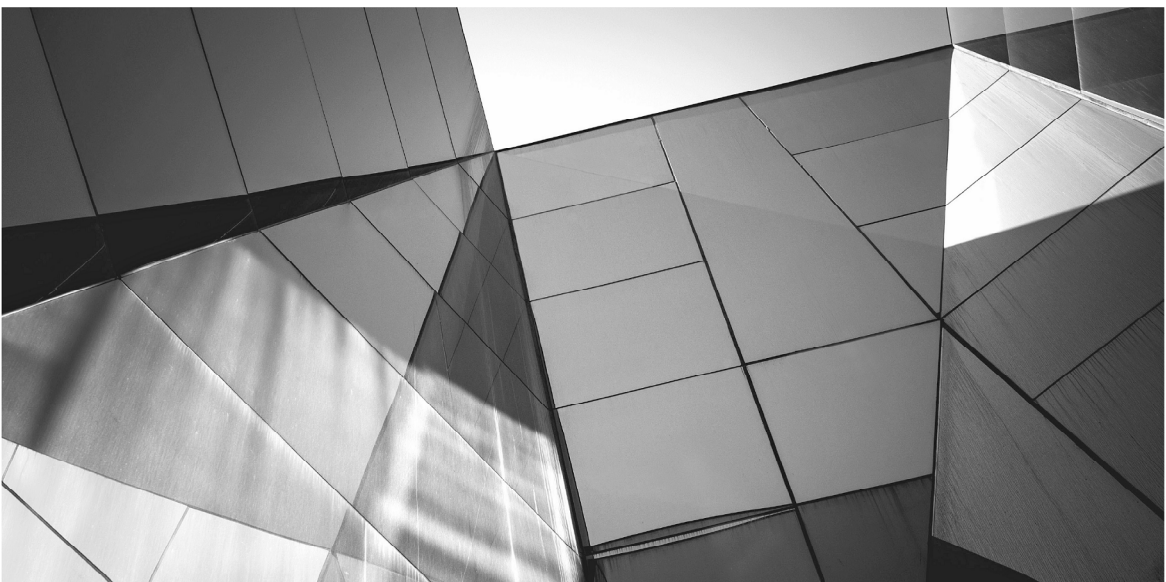
本书的所有章节由张龙翻译，参加本次翻译工作的还有张彤辉、焦伟、张淑贤、李志芹、张兴国、马元贺、王辉、王凯、单会明、周锐、王冠、高鹏、张淑华，在此一并表示感谢。

尽管在翻译过程中译者已经尽了最大的努力以确保译文的确和流畅，但囿于水平有限，读者在阅读过程中如果发现错误或不准确之处请及时与我联系，以便再版时修正。我的邮箱是 zhanglong217@163.com，博客是 <http://blog.csdn.net/ricohzhanglong>，新浪微博是@风中叶的思考，欢迎关注。最后，祝各位读者阅读愉快，早日掌握 lambda 表达式这一利器，编写出更加优雅的 Java 代码。



作者简介

Maurice Naftalin 在 IT 领域拥有 30 多年的经验，担任过开发者、设计师、架构师、经理、教师以及作者等角色。Naftalin 是经过认证的 Java 程序员，使用过 Java 的各个发布版本。他在 Java 与业务上的经历让他对 Java SE 8 中引入 lambda 表达式所带来的根本性变化有着独到的见解。Naftalin 是各种大会上的常客，包括一年一度的 JavaOne。他与 Oracle 开发团队协作运营着一个颇受欢迎的网站——www.lambdafaq.org，该网站主要关注于 Java 8 中的新语言特性。



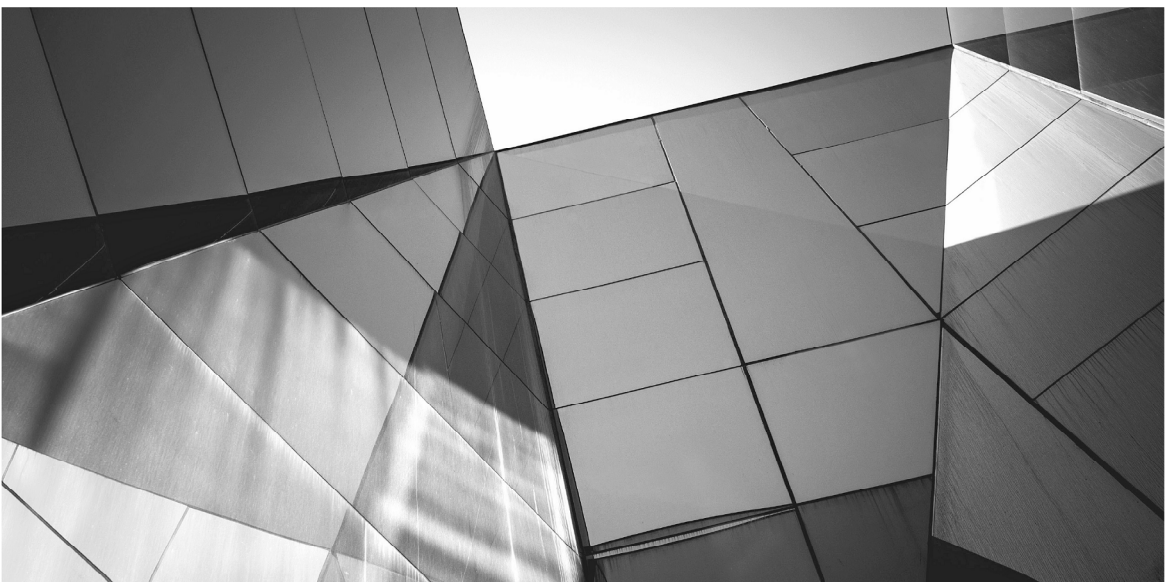
技术编辑简介

Stuart Marks 就职于 Oracle Java 平台组的 JDK 核心库团队。他目前从事的是 `lambda`、流与集合相关的工作，同时改进测试质量与性能。他之前在 Sun Microsystems 从事 JavaFX 与 Java ME 相关的工作。他在窗口系统、交互式图形以及移动和嵌入式系统领域有着 20 多年的软件平台产品开发经验。Stuart 拥有斯坦福大学计算机科学硕士学位以及电子工程学士学位。他目前与妻子和女儿居住在加利福尼亚。

Brian Goetz 是 Java 编程的权威之一。他是畅销书 *Java Concurrency in Practice* 的作者，此外还撰写了 75 篇之多的关于 Java 开发的文章。他是 JSR-335(Java 语言 `lambda` 表达式)规范的

VIII 精通 lambda 表达式：Java 多核编程

制定领头人，并且在很多 JCP 专家组中担任职位。Brian 是 Oracle 的 Java 语言架构师。

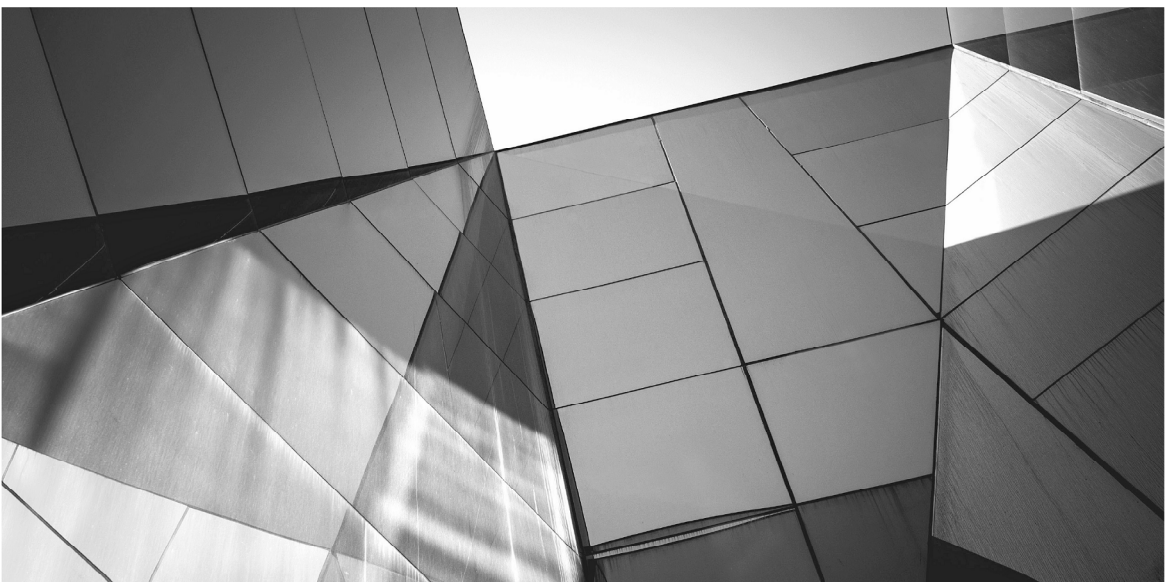


致 谢

如果没有来自于 Oracle 语言团队的伙伴们(Brian Goetz、Paul Sandoz、Aleksey Shipilev 与 Dan Smith)持续不断的帮助、鼓励与反馈,本书是不可能面世的。Stuart Marks 给出的建议非常有价值,对打磨本书起到了重要作用。

我要感谢 Graham Allan、Maurizio Cimadore、Chris Czarnecki、John Kostaras、Kirk Pepperdine、Jeremy Prime 与 Philip Wadler,他们的审校避免了很多错误的发生,同时经常给我指出新的方向。当然,任何遗留的错误都是我的责任。

非常感激本书的编辑 Brandi Shailer,感谢她在本书漫长的写作过程中给予我的耐心和乐观态度。



序 言

Java SE 8 是有史以来对 Java 语言和库改变最大的一次。既然你手头阅读的这本书的书名是《精通 lambda 表达式: Java 多核编程》，那么你可能已经知道了最大的新特性就是增加了 lambda 表达式。根据视角的不同，这种演变可能起始于 2009 年(那时启动了 Project lambda)，也可能起始于 2006 年(有几个提案希望 Java 增加闭包)，还有可能起始于 1997 年(引入了内部类)，甚至起始于 1941 年(Alonzo Church 发布了关于计算理论的基础研究工作成果，lambda 表达式这个名字就是从中得来的)。

无论多久，这都是关于时间的问题！虽然一开始 lambda 表达式似乎只是“另一个语言特性”而已，但实际上，它们会改变你

XII 精通 lambda 表达式：Java 多核编程

思考编程的方式，提供强大的新工具，将抽象应用到你每天都会面对的编程挑战中。当然，Java 已经提供了用于抽象的强大工具，例如继承与泛型等，但它们在很大程度上只是关于数据抽象的。lambda 表达式则提供了用于对行为进行抽象的更棒的工具来弥补这一点。

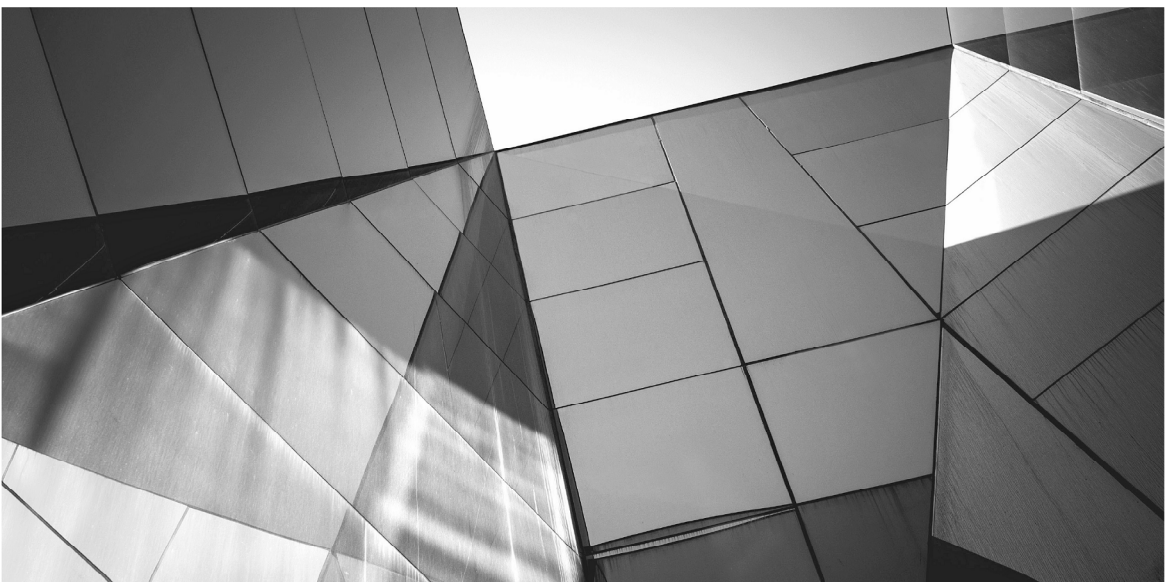
在拥抱 lambda 表达式的过程中，Java 针对函数式编程采取了一种温和的转变方式。虽然看起来面向对象编程与函数式编程之间是竞争关系，却为我们提供了互补的工具来管理程序的复杂性。随着硬件并行化持续升温，函数式编程的构建块(不变值与纯函数)甚至成为管理这种复杂性的更为有效的工具。

《精通 lambda 表达式：Java 多核编程》以分层、有序的方式详细介绍了 Java SE 8 中关于语言与库的新特性，包括 lambda 表达式、默认方法与 Streams 库等。更为重要的是，它将特性的细节与底层的设计决策联系起来，促使读者能够理解背后的动机与原则，从而最大限度地掌握这些新特性。与此同时，本书专注于真正的价值，这不是特性本身，而是特性所带来的好处：表述性更强、更为强大且不易出错的用户代码。让读者意识到这种好处是本书真正所关注的方面。

让本书指引你利用全新且改进的 Java 进行编程吧。一旦起步，我相信你就再也停不下来了！

——Brian Goetz

Oracle 公司 Java 语言架构师



前言

Java 8 可谓 Java 语言历史上变化最大的一个版本，其承诺要调整 Java 编程向着函数式风格迈进，这有助于编写出更为简洁、表达力更强，并且在很多情况下能够利用并行硬件的代码。在本书中，你将会发现引入 `lambda` 表达式这一表面上看起来细小的变化将如何使这一切成为可能。你将学习到如何通过 `lambda` 表达式使用一行代码编写 Java 函数，如何通过这种功能使用新的 `Stream API` 进行编程，如何将冗长的集合处理代码压缩为简单且可读性更好的程序。学习创建和消费流的机制，分析其性能，能够判断何时应该调用 `API` 的并行执行特性。

最后，为将新特性集成到现有的 Java 平台库中，需要对已有

的集合接口进行演化，而之前由于兼容性问题这一点是没法实现的。你将学习到如何通过默认方法来解决这些问题，如何在演化自己的 API 时使用它们。

第 1 章 走进新生代的 Java

本章为将 lambda 表达式与流引入到 Java 中做好了准备，其变化的动机是需要更好的编程模型以及让 Java 开始为多核处理器提供支持。

第 2 章 Java lambda 表达式基础

本章介绍了 lambda 表达式的语法，如何使用它们，在何处使用及其与匿名内部类的区别，以及由方法和构造器引用所提供的便捷缩写。

第 3 章 流与管道介绍

本章介绍了流的生命周期以及流编程的基础知识，提供了通过流源以及中间和终止操作处理集合的示例。

第 4 章 终止流：集合与汇聚

本章详细介绍了终止操作，特别是如何通过可变的汇聚操作将流元素汇聚到集合中。本章通过收集器(可变汇聚的库实现)扩展了第 3 章的示例。我们将会看到何时应该超越库实现的限制，以及如何编写自己的收集器。

第 5 章 起始流：源与分割迭代器

本章介绍了起始流的各种方式，包括使用库类，以及在必要时编写自己的分割迭代器。本章还深入介绍了流编程中的异常处

理。通过流处理重新实现 `grep` 的各种选项来展现出该模型的灵活性。

第 6 章 流的性能

本章介绍了如何确定并行执行的流处理的相对性能，方式是将源、中间操作的负载以及终止操作的并发性分割开来进行度量。此外还引入了微基准测试度量流的性能，同时还通过这些方式对书中的其他程序进行了分析。

第 7 章 使用默认方法来演化 API

本章介绍了新引入的默认方法是如何解决 Java 编程中长久以来存在的问题的，特别是如何首次使得基于接口的 Java API 的演化成为可能。本章还介绍了静态接口方法的使用。

本书读者对象

本书面向那些使用过 Java 5 及之前任意版本，同时又听说过 Java 8 中激动人心的变化，并且想要学习它们的 Java 开发者。你无须了解其他语言中的 `lambda` 表达式与闭包，也无须拥有函数式编程经验(当然，如果知道会更好)。

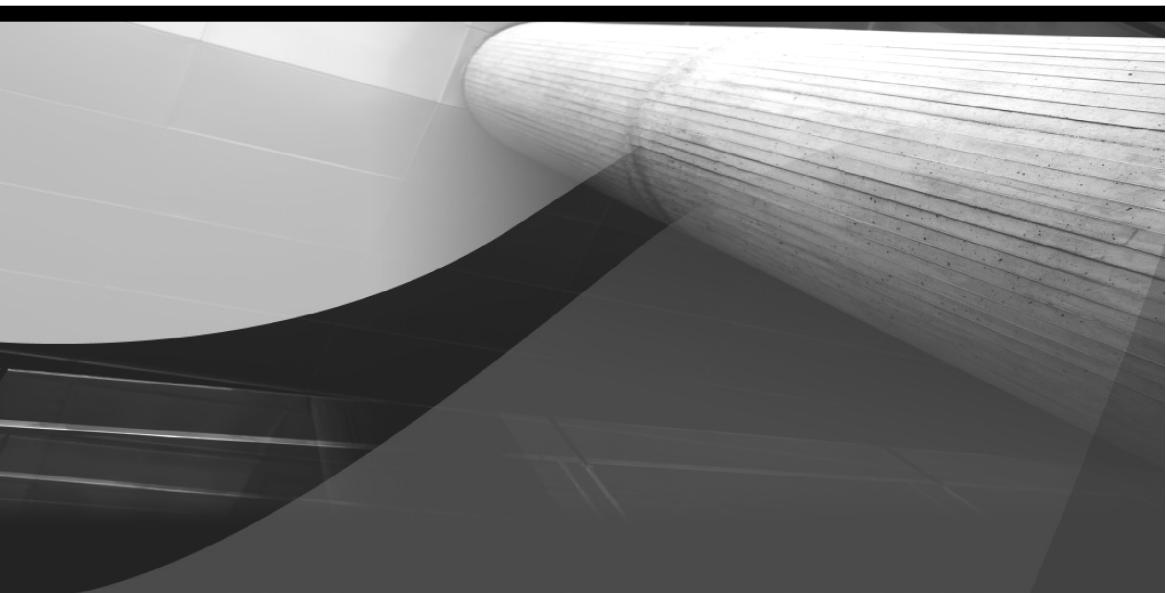
除了 Java 集合框架的标准集合外，本书不要求你熟悉其他的平台库，如果对标准集合不熟悉，请适时参阅 Javadoc 文档。

某些章节提供了一些高级主题：它们适合于延伸阅读。

示例、反馈与进一步学习

书中的代码可以从 Oracle 出版社的网站下载，网址是 www.OraclePressBooks.com。源代码与勘误也位于本书的产品页 www.mhprofessional.com。只需要搜索 ISBN 并下载必要的文件即可。

读者可以访问本书的支持网站 www.masteringlambdas.org 进行讨论、寻找进一步学习的链接，还可以联系作者。



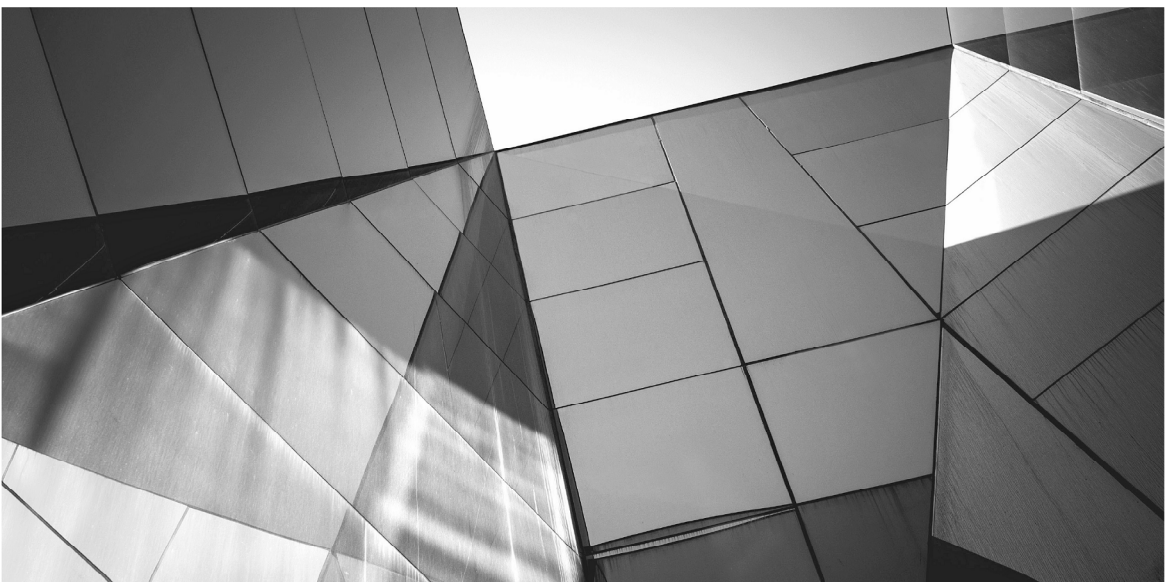
目 录

第 1 章	走进新生代的 Java	1
1.1	从外部迭代到内部迭代	2
1.1.1	内部迭代	4
1.1.2	命令模式	6
1.1.3	lambda 表达式	8
1.2	从集合到流	11
1.3	从串行到并行	15

1.4	组合行为	18
1.5	小结	22
第 2 章	Java lambda 表达式的基础知识	23
2.1	lambda 表达式的定义	24
2.2	lambda 与匿名内部类	26
2.2.1	无标识性问题	26
2.2.2	lambda 的作用域规则	27
2.3	变量捕获	29
2.4	函数式接口	32
2.5	使用 lambda 表达式	37
2.6	方法与构造器引用	39
2.6.1	静态方法引用	40
2.6.2	实例方法引用	41
2.6.3	构造器引用	44
2.7	类型检查	44
2.7.1	何为函数类型	45
2.7.2	匹配函数类型	46
2.8	重载解析	48
2.8.1	lambda 表达式的重载	49
2.8.2	方法引用的重载	52
2.9	小结	54
第 3 章	流与管道介绍	55
3.1	流基础	56
3.1.1	面向并行的代码	59
3.1.2	原生流	61
3.2	剖析管道	63
3.2.1	开始管道	63
3.2.2	转换管道	64

3.2.3	非侵入性	75
3.2.4	终止管道	78
3.3	小结	90
第 4 章	终止流：收集与汇聚	91
4.1	使用收集器	94
4.1.1	独立的预定义收集器	94
4.1.2	组合收集器	99
4.1.3	链接管道	104
4.1.4	示例说明：最流行的主题	106
4.2	剖析收集器	108
4.3	编写收集器	111
4.3.1	完成器	115
4.3.2	示例说明：找到我的书	118
4.3.3	收集器的规则	122
4.4	汇聚	124
4.4.1	对原生值的汇聚	124
4.4.2	对引用流的汇聚	126
4.4.3	通过汇聚来组合收集器	131
4.5	小结	132
第 5 章	起始流：源与分割迭代器	135
5.1	创建流	136
5.2	分割迭代器与 Fork/Join	145
5.3	异常	149
5.4	示例说明：递归 grep	155
5.5	小结	166
第 6 章	流的性能	167
6.1	微基准度量	170

6.1.1	度量动态运行时	171
6.1.2	Java Microbenchmarking Harness	173
6.1.3	试验方法	174
6.2	选择执行模式	178
6.3	流的特性	181
6.4	排序	184
6.5	有状态操作与无状态操作	187
6.6	装箱与拆箱	188
6.7	分割迭代器性能	189
6.8	收集器性能	190
6.8.1	并发 Map 的合并	190
6.8.2	性能分析：对点进行分组	192
6.8.3	性能分析：找到我的书	192
6.9	小结	194
第 7 章	使用默认方法演化 API	195
7.1	使用默认方法	199
7.2	抽象类的角色是什么	201
7.3	默认方法的语法	203
7.4	默认方法与继承	204
7.5	接口中的静态方法	211
7.6	小结	213
本书总结		215



第 1 章

走进新生代的 Java

Java 8 可谓 Java 语言历史上变化最大的一个版本，这体现在语言、库与虚拟机协调一致的变化上。其承诺要改变我们思考 Java 程序执行的方式，并且让语言适合于不久之后将要到来的在大规模并行硬件上的使用。不过相对于如此重要的创新，语言的实际变化却显得有些微不足道。这些表面上的微小变化到底是如何产生这种巨大的差别的呢？我们为何要改变 Java 中已经使用了这么多年(实际时间比这还要长)的编程模型呢？本章将会谈及这种模

2 精通 lambda 表达式：Java 多核编程

型的限制，同时还会介绍 Java 8 中 lambda 相关的特性是如何促使 Java 不断演进以满足新一代硬件架构的挑战的。

1.1 从外部迭代到内部迭代

首先来看一段简单的代码，这段代码会迭代一个包含着可变对象的集合，并对集合中的每个对象调用一个方法。如下代码片段会构造一个 `java.awt.Point` (Point 是个便捷、简单的库类，包含了一对(x,y)坐标)对象的集合。接下来，代码会对集合进行迭代，并对每个 Point 沿着 x 与 y 轴各平移 1 个单位的距离。

```
List<Point> pointList = Arrays.asList(new Point(1, 2), new
Point(2, 3));
for (Point p : pointList) {
    p.translate(1, 1);
}
```

在 Java 5 引入 for-each 循环前，我们可以像下面这样编写循环：

```
for (Iterator pointItr = pointList.iterator();
pointItr.hasNext(); ) {
    ((Point) pointItr.next()).translate(1, 1);
}
```

下面这种写法会更清晰一些(不过并不推荐这种写法，因为它扩大了 pointItr 的作用域)：

```
Iterator pointItr = pointList.iterator();
while (pointItr.hasNext()) {
    ((Point) pointItr.next()).translate(1, 1);
}
```

上述代码中，`pointList` 会创建一个 `Iterator` 对象，接下来我们通过该对象依次访问 `pointList` 中的元素。这个版本的代码还是非常有意义的，因为时至今日这正是 Java 编译器生成的用于实现 `for-each` 循环的代码。对于我们来说，其关键在于访问 `pointList` 中元素的顺序是由 `Iterator` 控制的——我们无法改变这一点。比如，针对 `ArrayList` 的 `Iterator` 会按照先后顺序返回列表中的元素。

这么做为什么会有问题呢？毕竟，Java Collections Framework 是在 1998 年设计的，看起来按照这种方式来指定列表元素的访问顺序是合情合理的。从那时起发生了什么变化呢？

部分原因在于硬件在不断演进。长久以来，工作站与服务器都配有多个处理器，不过在设计 Java Collections Framework 框架的 1998 年与个人电脑中首个双核处理器出现的 2005 年之间，芯片设计领域的革命爆发了。40 年来处理器速度以指数级别增长的趋势停止了，这是由于如下不可避免的事实造成的：信号泄漏、散热不充分，以及虽然到达了光速，但数据无法足够快地跨越芯片以实现更快的处理器速度的增长。

不过虽然存在时钟频率的限制，芯片组件的密度还在持续增加。因此，既然无法提供 6 GHz 的核，芯片厂商转而开始提供双核处理器，每个核运行于 3 GHz。这种趋势还在持续，目前还没有终止的迹象；在 Java 8 发布之时(2014 年 3 月)，四核处理器已经成为主流，八核处理器也已经出现在硬件市场上，而专业服务器早就在每个处理器中放置了几十个核。趋势很明显，不适应这种状况的任何编程模型都会在与适应这种状况的模型的竞争中败下阵来。适应意味着向开发者提供一种可访问的方式，使之能够将多个任务分发到多个核上并行执行，从而利用多核的处

理能力¹。另一方面，不适应意味着默认情况下 Java 程序会被绑定到单个核上，相对于那些能够帮助用户并行运行代码的语言来说，这会极大地降低程序的运行速度。

本节一开始的代码已经表明了改变的必要性，代码一次只能根据迭代器指定的顺序访问一个列表元素。集合处理并非程序员执行的唯一一个处理器密集的任务，不过它却是最重要的任务之一。Java 循环构建中所用的迭代模型会强制顺序处理集合元素，如果对运行时最迫切的需求刚好相反(至少要考虑性能因素)：将处理分发到多个核上，那么这就会产生严重的问题。第 6 章将会谈到，并不是每个问题都会从并行化中获益，不过最好的情况则是程序的加速几乎与核心数量线性相关。

1.1.1 内部迭代

在考虑将迭代的顺序模型应用到真实世界的场景中时就会发现其侵入性变得非常明显。如果有人通过如下指令让你邮寄一些信件——“重复如下动作：如果还有信件，那么按照收件人姓氏的字母表顺序取出下一个，然后将其放到邮箱中”，那么你可能会觉得他这么说太啰嗦了。你知道在这个任务中顺序并不重要，无论是顺序执行还是并行执行都可以，只不过不要遗漏信件即可。这时，你会觉得在存在更好策略的情况下，外部迭代导致集合只能连续并且按照固定顺序处理元素的做法实在是太低效了。

实际上，对于这个现实任务来说，你只需要知道每封信件都要邮寄出去即可；到底该怎么做取决于你自己。同样，我们应该告诉集合应该对它们所包含的每一个元素采取什么动作，而不是

¹ 将一个处理任务分发到多个处理器上通常称为并行化。即便我们不喜欢这个词，但这个简称还是很有用的，会让我们的解释更加简洁，可读性也更好。

像外部迭代那样指定怎么做。如果能够做到这一点，那么代码会变成什么样子呢？集合只需要公开一个方法来接受“做什么”，也就是说对每一个元素要执行什么任务；这个方法的一个显而易见的名字就是 `forEach`。借助于它，我们可以像下面这样替换掉本节一开始的迭代代码：

```
pointList.forEach(/*translate the point by (1,1)*/);
```

在 Java 8 之前，这个建议看起来会很奇怪，因为 `java.util.List` (`pointList` 的类型)并没有 `forEach` 方法，作为一个接口，我们也无法向其添加方法。不过，第 7 章将会看到 Java 8 通过引入非抽象接口方法解决了这一问题。

新方法 `Collection.forEach`(实际上是由 `Collection` 从其父接口 `Iterable` 继承的)只不过是内部迭代的一个示例，之所以这么说是因为，虽然已经看不到显式的迭代代码，但迭代依旧在内部发生。迭代现在由 `forEach` 方法管理，它会对集合中的每个元素应用其行为参数。

从外部迭代到内部迭代的变化看起来很小，只不过是迭代工作跨越了客户端-库的边界。不过，其结果却并不是那么简单。我们所需要的并行工作现在可以定义在集合类中，不必重复写在每一个要迭代集合的客户端方法中。此外，实现上可以自由使用其他技术，比如说延迟加载、乱序执行或是其他方法，从而更快地获得结果。

如果编程模型允许集合库的作者针对每个集合自由选择最佳的批处理实现方式，那么内部迭代就很有必要了。不过要想替换 `forEach` 调用中的注释，那么该如何告诉集合方法应对每个元素执行什么任务呢？

1.1.2 命令模式

没必要脱离传统的 Java 机制来寻觅这个问题的答案。比如说，我们会例行公事一般地创建 `Runnable` 实例并向其传递一些参数。如果你将 `Runnable` 当作代表了一个任务的对象，当 `run` 方法调用时就会执行任务，那么你会发现我们现在所需要的与之非常相似。再举一个例子，开发者可以通过 `Swing` 框架定义一个动作，该动作会执行以响应多种不同的用户界面事件，如菜单项选择、按钮按下等。如果熟悉经典的设计模式，那么你会发现这是对命令模式的描述。

在这种情况下，我们要考虑到底需要什么命令？我们的出发点是调用 `List` 中每个 `Point` 的 `translate` 方法。因此对于该例来说，`forEach` 应该将一个对象作为参数，该对象会公开一个方法，这个方法会对列表中的每个元素调用 `translate`。如果该对象是某个更加通用接口的实例，比如说 `PointAction`，那么就可以对 `Point` 集合迭代时所执行的不同行为定义不同的 `PointAction` 实现。

```
public interface PointAction {  
    void doForPoint(Point p);  
}
```

现在，我们需要的实现如下所示：

```
class TranslateByOne implements PointAction {  
    public void doForPoint(Point p) {  
        p.translate(1, 1);  
    }  
}
```

现在可以勾画出 `forEach` 的简单实现：

```
public class PointArrayList extends ArrayList<Point> {
```

```

    public void forEach(PointAction t) {
        for (Point p : this) {
            t.doForPoint(p);
        }
    }
}

```

如果让 `pointList` 成为 `PointArrayList` 的一个实例，那么就可以通过如下客户端代码实现内部迭代的目标：

```
pointList.forEach(new TranslateByOne());
```

当然，这段玩具代码针对性太强了；我们不应该为需要处理的每一种元素类型都编写一个新的接口。幸好，我们不需要这么做；名字 `PointAction` 与 `doForPoint` 没什么特别的；如果通过其他名字替换掉它们，那么结果不会有什么变化。在 Java 8 集合库中，它们称为 `Consumer` 和 `accept`。这样，`PointAction` 接口就变成了下面这样：

```

public interface Consumer<T> {
    void accept(T t);
}

```

参数化接口类型可以让我们省却专门的 `ArrayList` 子类，并且直接向类本身添加方法 `forEach`，正如 Java 8 继承所做的那样。该方法接收一个 `java.util.function.Consumer`，它会接收并处理集合中的每个元素。

```

public class ArrayList<E> {
    ...
    public void forEach(Consumer<E> c) {
        for (E e : this) {
            c.accept(e);
        }
    }
}

```

8 精通 lambda 表达式：Java 多核编程

```
    }  
}
```

将这些改变应用到客户端代码上，结果如下：

```
class TranslateByOne implements Consumer<Point> {  
    public void accept(Point p) {  
        p.translate(1, 1);  
    }  
}  
...  
pointList.forEach(new TranslateByOne());
```

你可能会觉得这段代码还是很笨拙。不过请注意，这种笨拙现在主要体现在类实例每个命令的表示上。在很多情况下，这有些过度了。比如说，在目前的情况下，`forEach` 真正所需的只是提供给它的对象的单个方法 `accept` 的行为。之所以包含进构成对象的状态和其他所有信息，是因为在 `Java` 中，如果不是原生数据类型，那么方法参数必须是对象引用。不过到目前为止，我们总是需要指定这些信息。

1.1.3 lambda 表达式

上一节的最后一段代码并非针对命令模式的惯用 `Java` 代码。在该例中，如果某个类很小并且不太可能重用，那么更加常见的做法是定义一个匿名内部类：

```
pointList.forEach(new Consumer<Point>() {  
    public void accept(Point p) {  
        p.translate(1, 1);  
    }  
});
```

有经验的 `Java` 开发者对上述代码会很熟悉，以至于我们都忘

记了初次遇到它时的感受。一般来说，初次遇到以这种方式使用的匿名内部类的语法的第一反应就是它太丑陋、冗长，并且难以快速理解，虽然它所做的全部事情只不过是“对每个元素进行该处理”。你不必完全同意这些判断，认为说服开发者使用这种方式来进行每一种集合操作是不可能的事情。我们只不过是通过它来引入 `lambda` 表达式而已²。

要想降低该调用的冗余性，我们应该识别出哪些地方提供的信息“编译器可以从上下文中推断出来”。其中一份信息就是匿名内部类所实现的接口名。编译器完全可以知晓 `forEach` 的参数类型是 `Consumer<T>`，足以通过这份信息来检查所提供参数的类型兼容性。我们不再强调编译器可以推断出的代码：

```
pointList.forEach(new Consumer<Point>() {  
    public void accept(Point p) {  
        p.translate(1, 1);  
    }  
});
```

其次，被重写的方法名是什么呢，在该例中是 `accept` 吗？一般来说，编译器是无法推断出此信息的。不过对于 `Consumer` 来说，无须推断其名字，因为该接口只有唯一一个方法。这种“单方法接口”模式对于定义回调来说是非常有用的，它甚至还有一个官方的表述：用于这种简写形式的任何对象都必须实现一个这样的接口，公开单个抽象方法(这称为函数式接口，有时也称为 SAM 接口)。这样，编译器就可以无歧义地选择出正确的方法。

² 人们经常好奇于 `lambda` 这个名字的由来。`lambda` 表达式的想法来源于美国数学家 Alonzo Church 于 20 世纪 30 年代提出的计算模型，其中希腊字母 λ (`lambda`)代表着函数抽象。不过为何会选择这个特殊的字母呢？Church 调侃到：问问他自己的选择吧，他的解释提到了排版错误，不过后来他又给出了另一个答案：“12345，上山打老虎”。

下面再次去除可以通过这种方式推断出的代码：

```
pointList.forEach(new Consumer<Point>() {  
    public void accept(Point p) {  
        p.translate(1, 1);  
    }  
});
```

最后，`Consumer` 的实例化类型通常可以从上下文推断出来，在该例中，当 `forEach` 方法调用 `accept` 时，它会提供给它一个 `pointList` 元素，该元素之前被声明为一个 `List<Point>`。这会将传递给 `Consumer` 的类型参数标识为 `Point`，从而让我们可以省略传递给 `accept` 的参数的显式类型声明。

下面就是去掉 `forEach` 调用的最后一部分不必要代码之后的内容：

```
pointList.forEach(new Consumer<Point>() {  
    public void accept(Point p) {  
        p.translate(1, 1);  
    }  
});
```

`forEach` 的参数代表一个对象，它实现了 `forEach` 所需的接口 (`Consumer`)，这样在对 `pointList` 元素 `p` 调用 `accept`(该接口的唯一一个抽象方法)时，其效果就等价于调用 `p.translate(1, 1)`。

这里要通过一个额外的语法(“`->`”)将参数列表与表达式体分隔开来。加上它之后，我们最终得到了一个简单形式的 `lambda` 表达式。下面就是用于内部迭代的代码：

```
pointList.forEach(p ->p.translate(1, 1));
```

如果从未使用过 `lambda` 表达式，那么你现在可以将其看作对方法声明的一种简写，将 `lambda` 参数列表映射为假想的方法的参

数列表，将 `lambda` 体(通常前面会有一个 `return`)映射为方法体。下一章将会看到，我们会改变之前示例所用的 `lambda` 表达式的简单语法，转而使用多个参数和更加复杂的 `lambda` 体，在这种情况下，编译器将无法推断出参数类型。不过，如果理解了其中的缘由，你就会对使用 `lambda` 表达式的动机及其所采取的形式有一个基本的认识。

本节已经介绍了很多。总结一下：我们首先考虑了编程模型所需做出的改变以适应变化的硬件架构的要求；然后回顾了集合元素的处理，这使得我们意识到需要有一种简洁的方式来定义待执行的集合的行为；最后，裁剪了匿名内部类定义的额外代码来让我们实现 `lambda` 表达式的简单语法。

在本章的剩余章节中，我们将会介绍 `lambda` 表达式实现的一些新的惯用语。我们将会以更加富有表现力的方式实现集合元素的批处理，这些惯用语上的变化会使得库编写者能够更加轻松地利用并行算法，充分发挥新硬件架构的优势，最后再介绍如何通过函数式行为改善 API 的设计。这种微小的改变可以获得巨大的成果！

1.2 从集合到流

我们来扩展一下上一节的示例。在现实的程序中，经常会通过多个步骤来处理集合：我们会迭代处理集合来生成新的集合，后者又会被迭代处理，以此类推。下面来看一个示例，该例是虚构出来的，并且做了一定的简化。该例首先生成了一个 `Integer` 实例的集合，接下来通过转换生成了一个 `Point` 实例的集合，最后寻找到距离原点最远的点到原点的距离。

12 精通 lambda 表达式：Java 多核编程

```
List<Integer>intList = Arrays.asList(1, 2, 3, 4, 5);
List<Point>pointList = new ArrayList<>();
for (Integer i : intList) {
    pointList.add(new Point(i % 3, i / 1));
}
doublemaxDistance = Double.MIN_VALUE;
for (Point p : pointList) {
    maxDistance = Math.max(p.distance(0, 0), maxDistance);
}
```

虽然这段代码还有改进空间，不过这却是大家惯用的 Java 代码——大多数开发者已经看到过很多这种模式的代码——不过如果以新人的眼光来看待它，那么你就会立刻发现一些令人讨厌的特性。首先，代码非常冗长，用了9行代码才实现这3个操作。其次，集合 `pointList` 只用作中间存储，但在程序操作中花费了很高的代价；如果中间存储非常大，那么往好的方面说，创建它会增加垃圾回收的成本，往坏的方面说则会导致可用的堆空间消耗殆尽。最后，这里有个难以发现的隐含假设，即空列表的最小值是 `Double.MIN_VALUE`。不过最糟糕的是开发者的意图与代码所表现出来的意图之间的巨大差异。要想理解该程序，你得搞清楚它做什么，然后猜测开发者的意图(如果幸运，还可以阅读注释)，然后通过匹配程序的操作与推理出的非正式规范来检查其正确性³。所有这些工作非常耗时间并且极易出错，事实上，高级语言的目标在于通过提供与开发者心智模型非常接近的代码来最小化这其中的工作量。那么该如何减小二者之间的沟壑呢？

下面来重述一遍问题：

“对 `Integer` 实例集合中的每一个元素应用变换来生成一个

³ 现在的情况要比过去好很多了。年龄稍大点的一些人还会记得使用汇编语言(低级语言，与机器码很接近)编写大型程序时的工作量。从那时开始，编程语言的表达力已经丰富了很多，不过还有很大的改进空间。

Point，然后找出距离原点最远的点与原点之间的距离”。

如果去掉上述代码中与该非正式规范不相关的部分，那么我们会发现代码与问题规范之间是非常不匹配的。去掉第一行代码(一开始创建列表 `intList` 的代码)，我们得到：

```
List<Point>pointList = new ArrayList<>();
for (Integer i : intList) {
    pointList.add(new Point(i % 3, i / 3));
}
doublemaxDistance = Double.MIN_VALUE;
for (Point p : pointList) {
    maxDistance = Math.max(p.distance(0, 0), maxDistance);
}
```

这展现出了一种新式的、面向数据的程序查看方式，如果了解 Unix 管道与过滤器，那么你就会对这种方式很熟悉：我们可以从源集合开始追寻单个值的处理，看着它首先由一个 `Integer` 转换为一个 `Point`，然后由一个 `Point` 转换为一个 `double`。这两个转换可以独立进行，无须引用其他被处理的值，这正是并行化的要求。只是对于第 3 步寻找最大距离来说需要值来进行交互(即便如此，我们也有方法来高效地进行并行计算)。

可以通过图形的方式来表达这种面向数据的视图，如图 1-1 所示。在该图中，矩形框代表操作。连接线代表将一个值序列发送到一个操作的新方式。这与任何类型的集合都不同，因为在给定的某个时刻，连接器所传递的值可能尚未产生。这些值序列称为流。流与集合不同，因为它提供了一个可选的有序值序列而无须为这些值提供任何存储；它们是“移动中的数据”，这是一种表示批量数据操作的方式。如果不熟悉流的概念，那么可以想象一种迭代器，其唯一的操作类似于 `next`，只不过除了返回下一个值之外，它还可以标识出后面已经没有值了。在 Java 8 集合 API 中，

流是通过接口表示的——Stream 代表引用值, IntStream、LongStream 与 DoubleStream 表示原生值的流——其位于包 java.util.stream 中。

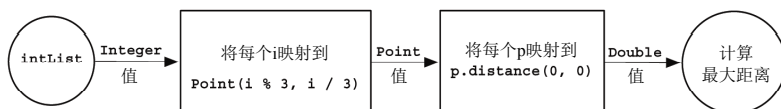


图 1-1 将流组合为数据管道

在这种视角下, 图 1-1 中的矩形框所表示的操作都是对流的的操作。图中的矩形框表示称为 **map** 的操作的两种应用; 它通过系统规则转换每一个流元素。单独查看 **map**, 我们会认为在操作单个流元素。不过, 很快就会看到其他流操作, 可以重排列、丢弃, 甚至插入值; 每个操作都可以描述为接收一个流, 然后以某种方式对其执行变换。每个矩形框都代表一个中间操作, 它不仅在流上定义, 而且还会返回流作为其输出。比如说, 假设一个流 **intStream** 构成了第 1 个操作的输入, 那么图 1-1 中间操作所执行的变换就可以通过代码表示为:

```
Stream<Point> points = intStream.map(i -> new Point(
    i % 3, i / 3));
DoubleStream distances = points.mapToDouble(
    p -> p.distance(0, 0));
```

管道最后的圆圈代表终止操作 **max**。终止操作会消费一个流, 并且有可能返回单个值, 如果流为空, 则什么都不返回, 这由一个空的 **Optional** 或是其专门的原生 **Optional** 之一来表示:

```
OptionalDouble maxDistance = distances.max();
```

图 1-1 中的管道有开始、中间处理以及结束。我们看到了定义在中间以及结束位置处的操作; 那么开始呢? 我们可以通过各

种源提供进入流中的值——集合、数组或是生成函数。实际上，常见的情况是将集合的内容提供给流，就像这里所做的那样。Java 8 集合针对该目标提供了一个新方法 `stream()`，这样管道的开始可以表示为：

```
Stream<Integer> intStream = intList.stream();
```

本节一开始的完整代码就变成了下面这样：

```
OptionalDouble maxDistance =  
    intList.stream()  
        .map(i -> new Point(i % 3, i / 3))  
        .mapToDouble(p -> p.distance(0, 0))  
        .max();
```

这种风格(通常称为流式风格，因为“代码在流动”)并不了解集合处理的上下文，一开始可能很难读懂。不过，相较于本节介绍的代码逐次迭代方式，它很好地平衡了简洁性与对问题声明之间的关系：“将源 `intList` 中的每个整数映射到相应的 `Point` 中，将结果列表中的每个 `Point` 映射到与原点之间的距离，然后找出结果中的最大值”。该代码结构突出了关键操作，而非像一开始那样模糊了这些操作。

作为奖励，创建与管理中间集合所导致的性能消耗也随之消失：顺序执行，流代码要比相应的循环版本快两倍。并行执行对于大型数据集来说会产生非常棒的加速效果。

1.3 从串行到并行

本章一开始就表示 Java 现在需要支持对集合的并行处理，而 `lambda` 则是提供这种支持的必备一环。我们已经看到客户端代码

通过 `lambda` 可以多么轻松地使用内部迭代。最后看到了对集合类的内部迭代是如何实现并行处理的。虽然不是每天都会用到，但了解其工作原理还是很有帮助的——对于客户端代码的开发者来说，其实现的复杂性实际上被隐藏了。

在多核上的独立执行是通过为每个核分配一个不同的线程来实现的，每个线程会执行整个工作的一个子任务，在该例中就是待处理的集合元素的一个子集。比如说，假如有一个四核处理器和包含了 N 个元素的一个列表，程序可以定义一个 `solve` 算法，将任务分解以实现并行执行，就像下面这样：

```
if the task list contains more than  $N/4$  elements {
    leftTask = task.getLeftHalf()
    rightTask = task.getRightHalf()
    doInParallel {
        leftResult = leftTask.solve()
        rightResult = rightTask.solve()
    }
    result = combine(leftResult, rightResult)
} else {
    result = task.solveSequentially()
}
```

上述伪代码非常简化地阐释了并行处理，它使用了名为递归分解的模式——递归地将大任务分解为小任务以并行执行，直到子任务变得“足够小”从而能串行执行为止。实现递归分解要了解如何以这种方式来分割任务，如何执行足够小的任务而不必再进一步分割，如何将这些小任务的执行结果合并到一起。到底该如何分割取决于数据源；在该例中，分割列表有着显而易见的实现。将子任务的执行结果合并起来通常是通过对其应用管道终止操作来实现的；对于本章的示例来说则是要得到两个子任务结果的最大值。

Java 的 `fork/join` 框架就应用了该模式，只不过它是从线程池中为新的子任务分配线程而不是创建新的线程。显然，重新实现该模式需要编写大量的代码，开发者每次处理一个集合时都得这么做，这并不是我们需要的。这是库应该做的事情！

对于该例来说，库类就是集合；在 Java 8 之前，集合库类可以像这样使用 `fork/join` 框架，这样客户端开发者就能以并行化的思维来解决业务问题了，这本质就是个性能问题。对于现在这个示例来说，客户端代码只需要做一处变更即可，如下代码所示：

```
OptionalDouble maxDistance =  
    intList.parallelStream()  
        .map(i -> new Point(i % 3, i / 3))  
        .mapToDouble(p -> p.distance(0, 0))  
        .max();
```

上述代码阐释了 Java 8 中引入并行化的意义所在：明确但不唐突。通过将最初的 `Integer` 值列表递归分解来实现并行执行，直到子列表变得足够小，然后串行执行整个管道，最后再通过 `max` 将结果合并起来，如上面的 `solve` 伪代码所示。到底什么是“足够小”取决于可用核的数量，有时还要考虑列表的性质。图 1-2 展示了如何根据四个核对列表进行分解；在该例中，“足够小”指的是将列表大小除以 4(与之相关的一个问题是确定什么样的列表是“足够大”的，从而值得采取并行执行的方式。第 6 章将会对这个问题进行详细的介绍)。

不唐突的并行化是 Java 8 的一个关键特性；API 的变化赋予了库开发者更大的自由。其中一个重要方面就是库开发者可以挖掘出现代以及未来机器架构所带来的性能改进。

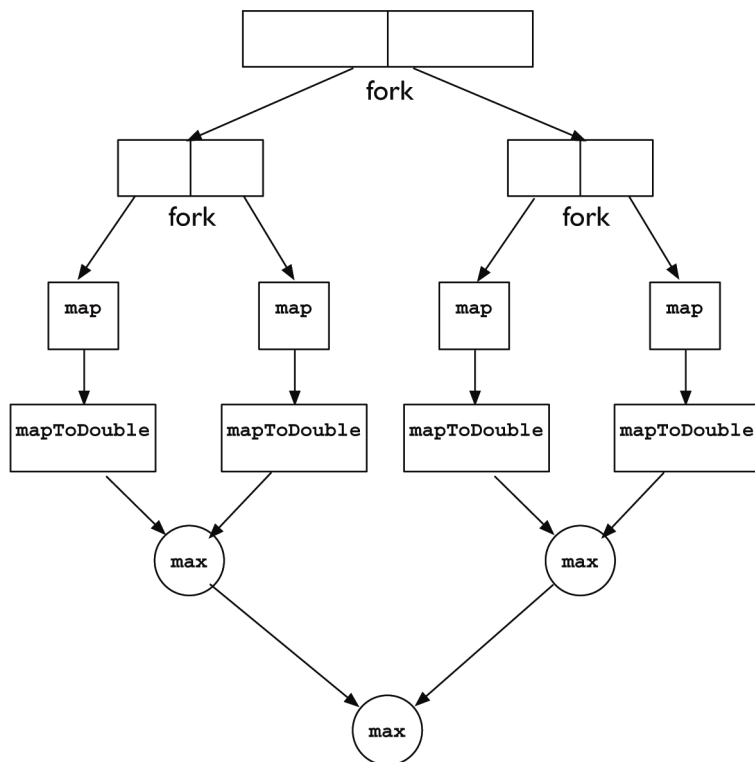


图 1-2 列表的递归分解处理任务

1.4 组合行为

在前面可以看到，从功能上来说，`lambda` 表达式与匿名内部类十分相似。不过它们二者的写法存在很大的差别，这也导致我们对其思考方式上的不同。`lambda` 表达式看起来像是函数，因此很自然地我们想知道是否可以让它们的行为像函数那样。这种视角上的转变让我们思考的是对行为而非对象的处理，反过来也会

造成编程方式与库 API 方向上的改变。

比如说，函数的一个核心操作是组合：将两个函数组合起来构成第 3 个函数，其效果等价于连续应用两个组件。组合并非匿名内部类的本性，不过从广义的形态来说，它非常类似于传统的面向对象编程。就像我们会通过解耦来分解面向对象程序一样，组合的反面也适用于函数。

假设我们想要按照 x 坐标对 `Point` 实例构成的列表进行排序。创建“自定义”排序⁴的标准的 Java 方式是创建一个 `Comparator`：

```
Comparator<Point> byX = new Comparator<Point>(){
    public int compare(Point p1, Point p2) {
        return Double.compare(p1.getX(), p2.getX());
    }
};
```

就像上一节那样，将匿名内部类声明替换为一个 `lambda` 表达式会提高代码的可读性：

```
Comparator<Point> byX =
    (p1, p2) -> Double.compare(p1.getX(), p2.getX());
```

❶

不过这么做对于另一个非常关键的问题爱莫能助：`Comparator` 是呆板的。如果想要定义一个比较 y 坐标而非 x 坐标的 `Comparator`，那就只能复制整个声明，将声明中的 `getX` 替换为 `getY`。好的编程实践促使我们寻求更好的解决方案，短暂的思考过后我们发现 `Comparator` 实际上执行的是两个函数——从参数中抽取出排序键，然后比较这些键。我们应该可以通过构建一个对

⁴ 在 Java 平台中，对象的比较与排序有两种标准方式：类可能会有自然顺序，在这种情况下，它会实现接口 `Comparable`，这样就会公开出一个 `compareTo` 方法，对象可以通过它将自身与其他对象进行比较。此外，还可以创建一个 `Comparator`，就像该例一样。

这两个组件参数化的 `Comparator` 函数来改进代码❶，现在就来做。这个中间过程看起来既笨拙又冗长，但请坚持下去：最后的结果还是值得我们这么做的。

首先，将已有的两个具体的组件功能转换为 `lambda` 形式。我们知道关键字抽取函数的函数接口类型(`Comparator`)，不过还需要知道与函数 `p -> p.getX()` 相对应的函数接口类型。查看专门为函数接口声明设计的包 `java.util.function`，我们会发现接口 `Function`：

```
public interface Function<T,R> {
    public R apply(T t);
}
```

现在就可以为关键字抽取与关键字比较编写`lambda`表达式了：

```
Function<Point,Double> keyExtractor = p ->p.getX();
Comparator<Double> keyComparer = (d1, d2) ->Double.compare(d1,
d2);
```

可以通过如下两个小函数来重新装配新版的 `Comparator<Point>`：

```
Comparator<Point> compareByX = (p1, p2) -> keyComparer.
compare(keyExtractor.apply(p1), keyExtractor.apply(p2)); ❷
```

这与❶的形式相匹配，不过却是一个重要的改进(这个改进在更复杂的情形下体现得更为明显)：你可以插入事先定义好的任何 `keyComparer` 或 `keyExtractor`。毕竟，我们的目标就是对构成更大的函数的小组件进行参数化。

虽然按照这种方式重新构建的 `Comparator` 改进了结构，但却失去了❶的简洁性。我们可以将其具体化，不过更为常见的情形则是 `keyComparer` 表达了对抽取的键的自然排序。那么❷可以重写为：

```
Comparator<Point>compareByX = (p1, p2) ->
    keyExtractor.apply(p1).compareTo(keyExtractor.apply(p2)); ❸
```

注意到该特例的重要性，平台库的设计者向接口 `Comparator` 添加了一个静态方法 `comparing`；给定一个关键字抽取器，它会使用针对键的自然顺序创建一个相应的 `Comparator`⁵。下面是其方法声明，其中的泛型类型参数已经进行了简化：

```
public static <T,U extends Comparable<U>>
    Comparator<T>comparing(Function<T,U>keyExtractor) {
    return (c1, c2) ->
        keyExtractor.apply(c1).compareTo(keyExtractor.apply(c2));
}
```

相对于❸，我们可以通过该方法编写如下代码(假设已经添加了对 `Comparators.comparing` 的静态导入声明)：

```
Comparator<Point> compareByX = comparing(p ->p.getX()); ❹
```

相对于❶来说，❹是一个重大的改进：更加简洁，更容易理解，这是因为它隔离并且强调了重要的元素(关键字抽取器)，之所以可以这样是因为 `comparing` 接受了一个简单的行为，并且通过它构建出更为复杂的行为。

为了能立刻看到改进，假设问题发生了一些变化，相对于找到距离原点最远的点，我们决定按照它们与原点之间的距离的升序将所有点打印出来。得到这种排序是轻而易举的事情：

```
Comparator<Point> byDistance = comparing(p ->p.distance(0, 0));
```

要想实现改变后的问题规范，我们只需要对流管道做小小的修改即可：

⁵ 还能以相同的方式为原生类型创建 `Comparator`，不过由于无法使用自然排序，因此它们会使用封装类提供的 `compare` 方法。

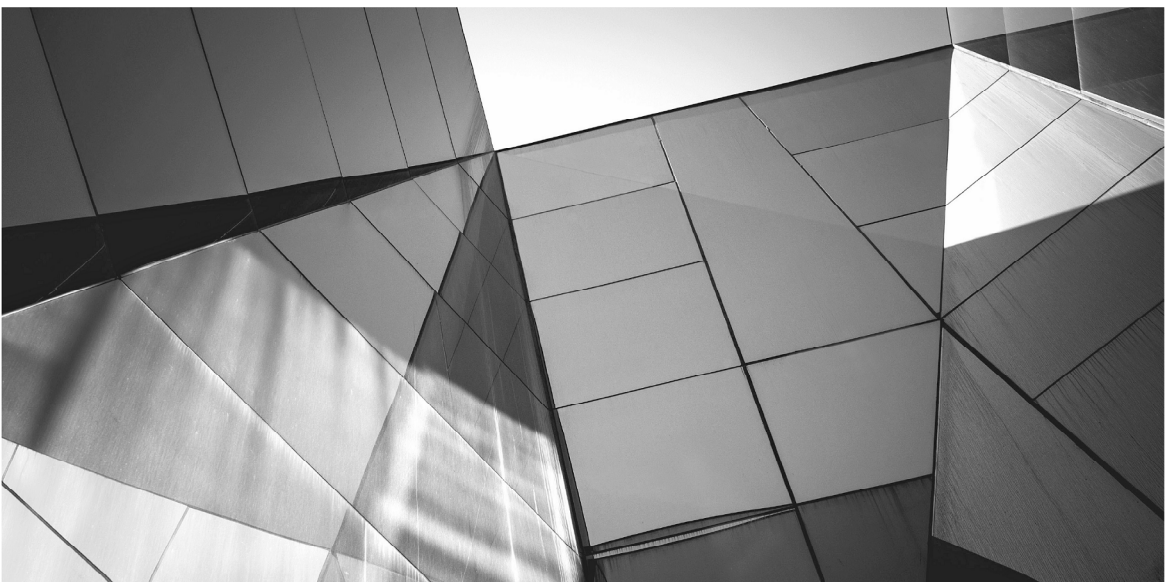
```
intList.stream()
    .map(i -> new Point(i % 3, i / 3))
    .sorted(comparing(p ->p.distance(0, 0)))
    .forEach(p ->System.out.printf("%f, %f", p.getX(),
p.getY()));
```

需要适应新的问题域的改变表明了 `lambda` 所带来的一些好处。改变 `Comparator` 是非常直接的，因为它是通过组合创建的，我们只需要指定改变的单个组件即可。新的比较器非常平滑地适应了现有的流操作，新的代码再一次非常接近于新的问题域，问题与代码之间变化的部分天然合一。

1.5 小结

现在，大家应该很清楚为何说 `lambda` 表达式是万众期待的了。在本章刚开始的小节中，我们看到了它们是如何提升性能的，库开发者可以通过 `lambda` 表达式实现自动的并行化。虽然这种改进未必是普遍的——本书的一个目标就是帮助你理解应用何时会从“并行化”中获益——但它代表了向正确方向前进的一大步，让应用程序员能够利用现代硬件的性能改进。

最后一节介绍了 `lambda` 是如何促使我们编写出更棒的 API 的。`Comparator.comparing` 的签名就是一个信号：随着客户端程序员逐步习惯于提供诸如 `comparing` 所接受的关键字抽取函数，`comparing` 这样细粒度的库方法将会成为标准，借助于此，客户端编码的风格将会得到改进，代码也会得到简化。



第 2 章

Java lambda 表达式的 基础知识

第 1 章简要介绍了 lambda 表达式以及将其引入 Java 中的动机。本章将会详细介绍 lambda 表达式的定义、如何将其应用于 Java 程序中，以及在什么地方应用。

2.1 lambda 表达式的定义

一般来说，在数学与计算中，lambda 表达式指的是一个函数：对于输入值的部分或全部组合来说，它会指定一个输出值。到目前为止，在 Java 中还没有办法编写独立的函数。我们常常使用方法来代替函数，不过它总是作为对象或类的一部分而存在。现在，lambda 表达式提供了一种与独立函数更为近似的方式¹。在传统的 Java 术语中，lambda 可以看成一种匿名方法，拥有更为简洁的语法，可以省略修饰符、返回类型、throws 语句，在某些情况下还可以省略参数类型。

lambda 的语法

Java 中的 lambda 表达式包含一个参数列表和一个 lambda 体，二者之间通过一个函数箭头 “->” 分隔。第 1 章的示例都包含了单个参数：

```
p -> p.translate(1, 1)
i -> new Point(i, i + 1)
```

不过与方法声明类似，lambda 可以接收任意数量的参数。除了像之前那样接收单个参数的 lambda 外，参数列表必须使用圆括号包围起来：

```
(x, y) -> x + y
() -> 23
```

¹ 关于 lambda 的一个常见问题就是它们是否是传统意义上定义在 Java 中的对象。这个问题并没有简单的答案，因为虽然 lambda 表达式会计算对象引用，但它们在其他方面的行为与对象则是完全不同的，请参见 2.2.1 节。

到目前为止，我们在声明参数时并没有显式指定类型，因为不指定类型时 `lambda` 的可读性通常会更好一些。不过，我们总是可以提供参数类型，有时这也是必要的，因为编译器可能无法从上下文中推断出其类型。如果显式提供了类型，那就必须为所有参数都提供类型，而且参数列表必须包围在圆括号中：

```
(int x, int y) -> x + y
```

可以像方法参数那样修改这种显式类型的参数，例如，可以将其声明为 `final`，也可以添加注解。

函数箭头右侧的 `lambda` 体可以是表达式，到目前为止所有示例都是这样的(注意，方法调用是表达式，包括那些返回 `void` 的方法)。诸如此类的 `lambda` 有时也称为“表达式 `lambda`”。更为一般的形式则是“语句 `lambda`”，其中的 `lambda` 体是一个块，也就是说，是由花括号包围的一系列语句：

```
(Thread t) ->{ t.start(); }  
() ->{ System.gc(); return 0; }
```

表达式 `lambda`:

```
args -> expr
```

可以看成相应的语句 `lambda` 的简写形式：

```
args -> { return expr; }
```

在块体中到底使用还是省略 `return` 关键字的原则与普通的方法体是一致的，也就是说，如果 `lambda` 体中的表达式有返回值，那就需要使用 `return`，也可以后跟一个参数来立刻终止 `lambda` 体的执行。如果 `lambda` 返回 `void`，那就可以省略 `return`，也可以使用它，但后面不带参数。

lambda 表达式不需要也不允许使用 `throws` 语句来声明它们可能会抛出的异常。

2.2 lambda 与匿名内部类

如果看过了第 1 章由匿名内部类到 lambda 表达式的转换，那么你可能想知道，除了具体语法外，二者之间真正的差别是什么。事实上，lambda 表达式有时被错误地称为匿名内部类的“语法糖”，这说的是二者之间只存在简单的语法上的变化。但实际上，二者之间存在很多显著差异，其中有两点对于程序员来说非常重要：

- 内部类创建表达式会确保创建一个拥有唯一标识的新对象，而 lambda 表达式的计算结果可能有，也可能没有唯一标识，这取决于具体实现。相对于对应的内部类来说，这种灵活性可以让平台使用更为高效的实现策略。
- 内部类的声明会创建出一个新的命名作用域，在这个作用域中，`this`与`super`指的是内部类本身的当前实例；相反，lambda 表达式并不会引入任何新的命名环境。这样就避免了内部类名称查找的复杂性，名称查找会导致很多小错误，例如想要调用外围实例的方法时却错误地调用了内部类实例的 `Object` 方法。

接下来的两节将会对这两点进行介绍。

2.2.1 无标识性问题

到目前为止，Java 程序的行为总是与对象相关联，以标识、状态和行为为特征。lambda 则违背了该规则；虽然它们会共享对象的一些属性，但其唯一的用处是表示行为。由于没有状态，因

此标识问题就不重要了。语言规范显式表示其是未确定的，唯一的要求就是 `lambda` 必须计算出实现了恰当函数接口(参见 2.4 节)的类实例。这么做的意图是赋予平台足够的灵活性来进行优化，如果每个 `lambda` 表达式都要拥有唯一标识，那么这种灵活性无法实现。

2.2.2 `lambda` 的作用域规则

就像大多数内部类一样，匿名内部类的作用域规则非常复杂，这是因为它可以引用从父类型继承下来的名字，以及声明在外部类中的名字。`lambda` 表达式则要简单得多，因为它们并不会从父类型中继承名字²。除了参数以外，用在 `lambda` 表达式体中的名字的含义与体外面是一样的。例如，像下面这样在 `lambda` 中再次声明一个局部变量就是非法的：

```
void foo() { final int i = 2; Runnable r = () -> { int i = 3; }}
// illegal
```

参数就像局部声明一样，因为它们可以引入新的名称：

```
IntUnaryOperator iuo = i ->{ int j = 3; return i + j; };
```

`lambda` 参数与 `lambda` 体局部声明可以隐藏字段名(也就是说，字段名可能会临时被重新声明为参数或局部变量名)。

```
class Foo {
    Object i, j;
    IntUnaryOperator iuo = i -> { int j = 3; return i + j; }
}
```

² 这个规则会将父类型(也就是函数接口)中声明的任何名字排除在 `lambda` 作用域之外。除了抽象方法外，接口可以声明静态的 `final` 字段、静态嵌套类以及默认方法(参见第7章)。它们都不在实现的 `lambda` 的作用域之内。

由于 `lambda` 声明就像简单的块一样，因此关键字 `this` 与 `super` 与外围环境的含义一样：也就是说，它们分别指的是外围对象及其父类对象。例如，如下程序会向控制台打印出两条 “Hello, world!” 消息：

```
public class Hello {  
    Runnable r1 = () -> { System.out.println(this); };  
    Runnable r2 = () -> { System.out.println(toString()); };  
  
    public String toString() { return "Hello, world!"; }  
  
    public static void main(String... args) {  
        new Hello().r1.run();  
        new Hello().r2.run();  
    }  
}
```

如果使用匿名内部类而非 `lambda` 表达式来编写同样的程序，那么它会打印出在内部类的对象上调用 `toString` 方法的结果。对于匿名内部类来说，更为常见的访问外围对象当前实例的用法要使用笨拙的语法 `OuterClass.this`，而这对于 `lambda` 来说是非常直接的。

关于如何解释 `this`：常常会有这样一个问题：`lambda` 能否引用自身呢？如果名字在作用域中，那么 `lambda` 就可以引用自身，不过初始化器中的前向引用限制规则（对于局部变量与实例变量均如此）导致 `lambda` 变量无法初始化。我们还是可以声明一个递归定义的 `lambda`：

```
public class Factorial {  
    IntUnaryOperator fact;  
    public Factorial() {  
        fact = i -> i == 0 ? 1 : i * fact.applyAsInt(i - 1);  
    }  
}
```

需要递归的 `lambda` 定义的情况并不太多，这种做法完全可以胜任该任务。

2.3 变量捕获

上一节介绍了如何对 `lambda` 表达式从其外围环境中继承下来的名字进行解释。不过名字解释只是一部分；一旦理解了从环境中继承下来的变量名的含义，我们还需要知道能对它们做什么，应该做什么，这是两件不同的事情。

首先，我们发现很多实用的 `lambda` 表达式实际上并不会从其环境中继承任何名字。面向对象程序员可以通过与静态方法做类比来理解这一点；虽然一般来说，对象的行为依赖于其状态，但很多时候我们定义的方法并不会依赖于系统状态。例如，辅助类 `java.lang.Math` 只包含了静态方法，在计算数字的平方根时考虑系统状态是毫无意义的事情。`lambda` 可以承担同样的角色，与调用 `Math.sqrt` 得到相同结果的 `lambda` 如下代码所示：

```
DoubleUnaryOperator sqrt = x -> Math.sqrt(x)
```

像这种只通过参数和返回值与环境进行交互的 `lambda` 称为无状态或非捕获 `lambda`。与之相反，捕获 `lambda` 会访问外围对象的状态。“捕获”是个技术术语，指的是保留住 `lambda` 对其环境的引用。其含义表示变量已经被 `lambda` 引用到，可以对其进行查询；在其他语言中，后面在计算该 `lambda` 时还可以修改该变量。

捕获所提供的访问能力是有限制的；这种限制的中心原则就是捕获变量的值不能修改。因此，虽然传统叫法是“变量捕获”，不过事实上称为“值捕获”更为准确一些。为了解该原则是如

何实现的，我们首先来考虑局部变量捕获；接下来再看看字段捕获，这样就更好理解了。

传统上，一般对于局部类，特别是匿名内部类来说，要想在内部类中访问到外部方法中的局部变量，这些局部变量需要声明为 `final`。该规则也非常类似于 Java 8 中的 `lambda`，只不过语法上的要求没那么严格而已：要想确保 `lambda` 不会修改从外部环境中捕获到的变量值，该变量必须定义为 `final`，这意味着变量在初始化后不会在任何地方被赋值(从 Java 8 开始，匿名类与局部类也可以访问到 `final` 变量)。

本质上，对于那些被当作 `final` 的变量来说，我们可以省略声明中的 `final` 关键字。相比于其他语言，Java 中只能对 `final` 变量进行捕获的限制引起了很多争论，例如 JavaScript 就没有这个限制。防止 `lambda` 修改局部变量的理由是这么做会引入非常复杂的变化，而这些变化会影响到程序的正确性与性能，无论如何，这么做都是毫无必要的：

- **正确性**：放开这个限制会引入一类与局部变量相关的多线程Bug。在Java中，到目前为止局部变量都不会产生竞态条件与可见性问题，这是因为它们只能由执行局部变量所在方法的线程访问到。不过我们可以将`lambda`从创建它的线程传递给其他线程，这样如果第2个线程中的`lambda`可以修改局部变量，那么方才提到的竞态条件与可见性问题就会出现。

此外，无论涉及多少个线程，`lambda`的生命周期可能会大于对计算它的方法的调用。如果捕获的局部变量是可变的，那么其生命周期就得比创建它们的方法调用长。抛开其他后果不谈，这种改变会引入与局部变量相关的内存泄漏问题。

- **性能**: 如果对变量的访问是同步保护的, 那么允许多线程访问可变变量的程序的正确性就是可以保证的。不过这么做的代价却不太符合引入 `lambda` 的一个主要目标——将对不同参数的函数的计算分发到不同线程上这一策略。甚至从不同线程中读取可变的局部变量的值都得引入同步或是使用 `volatile`, 从而避免读取到旧数据。
- **非根本性**: 审视该限制的另一种方式就是考虑它不允许使用的场景。该限制禁止使用的惯用法涉及初始化与变化, 就像下面这个对一个 `List<Integer>` 中的值求和的示例:

```
int sum = 0;
integerList.forEach(e -> { sum += e; }); // illegal
```

`Stream API` 提供了更棒的选择。对于这个简单的示例来说, 我们可以写成:

```
int sum = integerList.stream()
    .mapToInt(Integer::intValue)
    .sum();
```

后续章节将会详细介绍, `Java 8` 设计的一个指导原则就是学习这种函数式风格要比其所带来的代码质量上的改进更为重要。这种风格的代码也是面向并行的(参见 3.1.1 节), 这可以看作一个额外的好处, 在目前的某些情况下它会带来更棒的性能, 未来这种情况将会更多。

其实对 `final` 的限制很容易就能规避, 这也不是什么秘密了。例如, 如果局部变量是个数组引用, 变量是 `final` 的, 不过数组内容还是可变的。可以通过这个众所周知的技巧实现迭代处理, 不过在并行执行时, 你很可能无意间就造成了竞态条件。可以通过同步来防止竞态条件的出现, 不过这么做会导致竞争加剧并降低

性能³。一言以蔽之，不要这么做！

看起来对字段名的捕获没有诸如 `final` 之类的限制，但实际上与局部变量是一样的：对字段名 `foo` 的引用实际上是 `this.foo` 的简写，其中的伪变量 `this` 担负着不可变的局部变量的角色。正如对其他对象字段的引用一样，如果 `lambda` 就是对象引用，那么在该示例中所捕获到的值就是 `this`。在计算 `lambda` 时，`this` 会被解引用(就像对任何局部引用变量一样)，字段会被访问。

似乎允许修改局部变量的呼声也应该出现在字段上。不过 `lambda` 打算提供一种更加温和的方式，有更好的实现方式而不是将 Java 彻底转变成函数式语言。在 Java 8 之前，修改共享变量是很轻松的事情，非常容易！开发者的职责就是避免这么做，如果可能，还要将其管理起来。通过 `lambda` 来修改字段值也没有改变这一状况。

2.4 函数式接口

从 1.1.3 节我们知道，`lambda` 表达式必须实现一个函数式接口。不过，要想实际使用 `lambda` 与函数式接口，我们需要更准确地理解二者之间的关系。本节将会概览并简要介绍库 `java.util.function` 所提供的函数式接口。稍后(2.7 节)将会更详尽地介绍二者之间的关系。

函数式接口之所以起到这种核心作用源自于其最为重要的一个属性，这个属性赋予了函数式接口的名字：它们可用于描述函数的类型。例如，接口 `UnaryOperator`：

³ 更安全的做法也是存在的，例如 `AtomicInteger` 或 `LongAdder`。不过更好的做法则是在尽可能的情况下不要同时修改共享变量。


```
public interface UnaryOperator<T> { T apply(T t); }
```

描述了如下函数：

```
f: T -> T
```

这是其函数类型，也是最为简单、最为常见的情况，仅仅是函数式接口的单个抽象方法的方法类型：即方法的类型参数、形参类型、返回类型，以及在适用情况下的抛出类型(函数类型之前称为“函数说明符”；你可能还会遇到这个术语)。2.7.1 节将会更为详尽地介绍函数类型。

函数类型是 `lambda` 必须匹配的，这包括通过装箱或拆箱、增大或缩小范围等手段实现的类型适配。例如，假设我们将变量 `pointList` 声明为 `List<Point>`，现在想要替换里面的每一个元素。方法 `replaceAll` 就可以很好地实现这个目标：

List<E>	
<code>replaceAll(UnaryOperator<E>)</code>	<code>void</code>

类图约定

本书中的平台库 API 的类图采用了一些简化的缩写：

- 右上角的图标包含了“i”或“s”，表示类图是否包含实例方法或静态方法。
- 省略泛型参数类型的通配符边界(例如，`forEach` 的参数实际上是 `Consumer<? extends T>`)。
- 如果发现方法声明中的类型变量没有出现在类声明中，那就可以假定它是方法的类型参数。
- 类图不一定是完整的，只会列出对于讨论来说重要的方法。

调用可以像下面这样：

```
pointList.replaceAll((Point p) -> { /* return new Point object
*/ });
```

要想编译上述代码，lambda 表达式需要匹配 `UnaryOperator<Point>` 的函数类型，它是如下方法的类型：

```
public Point apply(Point p);
```

该例直观易懂，不过一般来说，匹配过程会对类型检查提出一些挑战。之前，任何结构良好的 Java 表达式都会有一个定义类型；现在，虽然类型良好的语句或表达式中的每个 lambda 都会实现一个函数式接口，但 lambda 本身只会部分决定它到底实现了哪个函数式接口。上下文需要提供足够多的信息才能够推断出确切的类型，这个过程称为目标类型⁴。下面这个示例展示的 lambda 表达式就拥有多种可能的类型：

```
x -> x * 2
```

该表达式可以是多种函数式接口的实例，包括声明在 `java.util.function` 中的下面两个。

```
public interface IntUnaryOperator {
    int applyAsInt(int operand);
}

public interface DoubleUnaryOperator {
```

⁴ 实际上，目标类型并不是全新的概念，不过它在 Java 8 中广泛用于编译 lambda 表达式，这与传统 Java 程序的类型检查是不同的。现在，表达式的类型会先被计算出来，然后再根据其上下文进行兼容性检查；如果上下文是一个方法调用，那么恰当的重载就会作为上下文。与之相反，无类型的 lambda 表达式并没有要与上下文比对的类型，这样在方法调用的情况下，首先就要选择重载，不过依然要与 lambda 保持兼容。2.7 与 2.8 节将会介绍对编译器提出的这种挑战。

```
double applyAsDouble(double operand);
}
```

这样，如下两个赋值：

```
IntUnaryOperator iuo = x -> x * 2;
DoubleUnaryOperator duo = x -> x * 2;
```

都是合法的，因为每种情况下 lambda 表达式的类型都与目标类型兼容，也就是被赋值变量的函数式接口类型。编译器通过将 lambda 参数当作 int 类型来确定第一种情况的类型，这样 lambda 整体就是从 int 到 int 的函数。这与 IntUnaryOperator 的函数类型匹配，它也接收一个 int 并返回一个 int。与之类似，在第二种情况下，lambda 参数可以当作 double 类型，这样 lambda 就是从 double 到 double 的函数了，它与 DoubleUnaryOperator 的函数类型相匹配。

这两个 lambda 表达式从字面上看是一样的，就像我们之前看到的那样，不过它们的类型不同，含义也不同；对 int 值的操作与对 double 值的操作是不同的。类型是在编译期建立起来的，并且不能改变，因此我们无法为不同的类型重用相同的 lambda 表达式(除非它们可以通过类型转换保持兼容)，下一节将会介绍相关示例。

表 2-1 展示了 java.util.function 中 4 种基本的函数式接口类型，并且给出了示例用例以及 lambda 实例(其中的类型参数 T 被实例化为 String，U 被实例化为 Integer)。

java.util.function 中定义的 40 多个奇怪的类型都是通过 3 种不同路由的各种组合由这 4 个类型演化而来的：

表 2-1 java.util.function 中基本的函数式接口

接 口	参数 类型	返回 类型	示 例 用 例	示 例
Consumer<T>	T	void	使用对象过滤值工厂方法进行转换或是从对象中选择	s->System.out.print(s)
Predicate<T>	T	boolean		s->s.isEmpty()
Supplier<T>	无	T		()-> new String()
Function<T,U>	T	U		s-> new Integer(s)

- 原生特化：这些接口使用原生类型替换掉类型参数。代码如下：

```
interface LongFunction<R> { R apply(long value); }
interface ToIntFunction<T> { int applyAsInt(T value); }
interface LongToIntFunction { int applyAsInt(long value); }
```

- Consumer、Predicate 与 Function 的函数类型都接收单个参数。有接收两个参数的相应接口，代码如下：

```
interface BiConsumer<T,U> { void accept(T t, U u); }
interface BiFunction<T,U,R> { R apply(T t,U u); }
interface ToIntBiFunction<T,U> { int apply(T t, U u); }
```

- Function 的常见用法要求其参数与结果拥有相同的类型。之前在 List.replaceAll 的参数中已经看到了这一点。我们可以通过将 Function 的变种特化为相应的 Operator 来达成所愿，如下所示：

```
interface UnaryOperator<T> extends Function<T,T> { ... }
interface BinaryOperator<T> extends BiFunction<T,T,T> { ... }
interface IntBinaryOperator { int applyAsInt(int left, int
right); }
```

这个库是“入门套件”，旨在涵盖函数式接口的常见用例。如果你的用例没有涵盖，那么可以很轻松地声明自己的函数式接口，不过最佳实践则是在可能的情况下使用库中的函数式接口。此外，使用 `@FunctionalInterface` 来注解自定义的函数式接口声明也是最佳实践，这样编译器就可以检查接口只会声明一个抽象方法，同时相应的 Javadoc 也会自动添加说明部分。

2.5 使用 lambda 表达式

当对函数式接口实例的引用有效时就可以使用 `lambda` 表达式，前提是上下文提供了恰当的目标类型，也就是说，上下文明确需要一个函数式接口类型。例如，如下声明的目标类型：

```
IntPredicate ip = i -> i > 0;
```

是 `IntPredicate`，这是一个函数式接口，其函数类型兼容于 `lambda` 表达式 `i -> i > 0`。相反，声明：

```
Object o = i -> i > 0; // invalid
```

就无法通过编译，因为上下文所需的目标类型并不是函数式接口类型，因此它无法给编译器提供编译该 `lambda` 时所需的类型信息。

有 6 种上下文可以提供恰当的目标类型：

- 方法或构造器参数，在这种情况下，目标类型就是恰当的参数类型。第 1 章已经介绍过这类示例。
- 变量声明与赋值，在这种情况下，目标类型就是被赋值的类型。

38 精通 lambda 表达式: Java 多核编程

```
Comparator<String> cc =  
    (String s1, String s2) -> s1.compareToIgnoreCase(s2);
```

数组初始化器与之类似, 只不过其目标类型是数组组件的类型:

```
IntBinaryOperator[] calculatorOps = new IntBinaryOperator[]{  
    (x,y) -> x + y, (x,y) -> x - y, (x,y) -> x * y, (x,y) -> x / y  
};
```

lambda 数组应用范围有限, 不过, 由于大多数函数式接口都是泛型的, 因此泛型数组的创建是不允许的。

- 返回语句, 在这种情况下, 目标类型是方法的返回类型:

```
Runnable returnDatePrinter() {  
    return () -> System.out.print(new Date());  
}
```

- lambda 表达式体, 在这种情况下, 目标类型是 lambda 表达式体所期望的类型, 它由外层目标类型推导而来。考虑如下代码:

```
Callable<Runnable> c = () -> () -> System.out.println("hi");
```

这里的外层目标类型是 `Callable<Runnable>`, 其函数类型就是如下方法的类型:

```
Runnable call() throws Exception;
```

因此, lambda 体的目标类型就是 `Runnable` 的函数类型, 即 `run` 方法的类型。它不接收参数, 不返回值, 因此与内部的 lambda 匹配。

- 三元条件表达式, 在这种情况下, 两边的目标类型都由上下文提供。例如:

```
Callable<Integer> c = flag ? () -> 23 : () -> 42);
```

- 类型转换表达式，它会显式提供目标类型。例如：

```
Object o = () -> "hi";           // illegal
Object s = (Supplier) () -> "hi";
Object c = (Callable) () -> "hi";
```

第 1 个声明是不合法的，因为没有合适的目标类型能够解决含义模糊的 `lambda` 表达式声明。第 2 个与第 3 个是合法的，因为类型转换提供了目标类型。该例还表明字面上相同的 `lambda` 可能会有不同的类型；像下面这样尝试通过不同的类型重用 `lambda`：

```
Callable c1 = (Callable) s;
```

可以编译通过，不过在运行时抛出 `ClassCastException` 异常。

2.6 方法与构造器引用

我们已经知道，一般来说，任何 `lambda` 表达式都可以看作声明在函数式接口中的单个抽象方法的实现。不过，当 `lambda` 表达式只是调用现有类中的具名方法的一种方式时，编写 `lambda` 的更好方式则是使用已有的名字。例如，考虑如下代码，它会向控制台输出列表中的每个元素：

```
pointList.forEach(s -> System.out.print(s));
```

这里的 `lambda` 表达式只是将参数传递给 `print` 调用。诸如此类的 `lambda` (其唯一目的就是提供参数给一个具体方法)完全是由该方法类型定义的。因此，假如可以通过某种方式确定出类型，那么只包含方法名的简短形式所提供的信息就与完整的 `lambda`

表达式一样，但可读性会更好。相比于上述代码，我们可以这样编写：

```
pointList.forEach(System.out::print);
```

它表示相同的含义。这种对现有类的具体方法的操作写法称为方法引用。有 4 种类型的方法引用，如表 2-2 所示。本节后续部分将会介绍每种类型的目的。

表 2-2 方法引用类型

名 字	语 法	相应的 lambda 表达式
静态	RefType::staticMethod	(args) -> RefType.staticMethod(args)
绑定实例	expr::instMethod	(args) -> expr.instMethod(args)
未绑定实例	RefType::instMethod	(arg0,rest) -> arg0.instMethod(rest)
构造器	ClsName::new	(args) -> new ClsName(args)

2.6.1 静态方法引用

静态方法引用的语法只需要类与静态方法名，中间通过两个冒号分隔。例如：

```
String::valueOf
Integer::compare
```

是对静态方法的引用。为了理解如何使用静态方法引用，假设我们想要根据大小对一个 `Integer` 数组排序，数组中的每个值都被看成无符号的。`Integer` 的自然顺序根据数字来排序(也就是考虑到值的符号)，因此我们需要提供一个显式的 `Comparator`。我们可以使用静态方法 `Integer.compareUnsigned`：


```
(x,y) -> Integer.compareUnsigned(x, y);
```

这样，对数组 `integerArray` 排序就可以调用：

```
Arrays.sort(integerArray, (x,y) -> Integer.compareUnsigned(x,
y));
```

这是合法的，不过这么做要比相应的静态方法引用冗长和重复：

```
Arrays.sort(integerArray, Integer::compareUnsigned);
```

事实上，该方法是在 Java 8 中引入的，并且就是期望按照这种方式使用。未来，API 设计的一个要素就是希望方法签名要适合于函数式接口转换。

注意表 2-2 中的语法 `ReferenceType::Identifier` 并不总是代表对静态方法的一个引用。后面将会看到，该语法还可用于引用实例方法。

2.6.2 实例方法引用

有两种方式可以引用实例方法。绑定方法引用类似于静态引用，只不过是通过对 `ObjectReference::Identifier` 替换 `ReferenceType::Identifier`。之前的示例就是绑定方法引用：`forEach` 方法用于将集合中的每个元素传递给 `PrintStream` 对象 `System.out` 的实例方法 `print` 进行处理，如下 lambda 表达式：

```
pointList.forEach(p ->System.out.print(p));
```

可以替换为绑定方法引用：

```
pointList.forEach(System.out::print);
```

之所以称为绑定引用，是因为接收者已经确定为方法引用的

一部分。对方法引用 `System.out::print` 的每次调用都会有相同的接收者：`System.out`。不过，你常常会在调用方法引用时带上方法接收者及其参数(来自于方法引用的参数)。要想做到这一点，你需要一个未绑定的方法引用，之所以起这个名字，是因为接收者是不确定的；方法引用的第一个参数被用作接收者。在只有一个参数的情况下，未绑定方法引用是最容易理解的；例如，要想通过工厂方法 `comparing` 创建一个 `Comparator`，我们可以将如下 lambda 表达式：

```
Comparator personComp = Comparator.comparing(p ->
p.getLastName());
```

替换为未绑定方法引用：

```
Comparator personComp =
Comparator.comparing(Person::getLastName);
```

未绑定方法引用可以通过其语法识别出来：与静态方法引用一样，我们也使用格式 `ReferenceType::Identifier`，不过这里的 `Identifier` 指的是实例方法而非静态方法。要想探寻绑定与未绑定方法引用之间的差别，考虑调用方法 `Map.replaceAll` 并提供绑定与未绑定的实例方法引用：

Map<K,V>	
replaceAll(BiFunction<K,V,V>)	void

`Map.replaceAll` 的效果是对 `map` 中的每个键值对应用其 `BiFunction` 参数，并使用结果替换键值对中的值部分。如果变量 `map` 指向的是一个 `TreeMap`，并且其字符串表示如下：

```
{alpha=X, bravo=Y, charlie=Z}
```

那么像下面这样通过绑定方法引用来调用 `replaceAll`:

```
String str = "alpha-bravo-charlie";
map.replaceAll(str::replace)
```

效果就相当于三次应用 `str.replace`，即：

```
str.replace("alpha", "X")
str.replace("bravo", "Y")
str.replace("charlie", "Z")
```

每次调用的结果都会替换相应的值，执行完毕后 `map` 将包含：

```
{alpha=X-bravo-charlie, bravo=alpha-Y-charlie,
charlie=alpha-bravo-Z}
```

现在使用 `map` 的初始值来重新执行该例，再次调用 `replaceAll`，这次使用未绑定方法引用 `String::concat`，这是对 `String` 实例方法的引用，接收单个参数。使用单个参数的实例方法作为 `BiFunction` 看起来有些奇怪，不过事实上它是 `BiFunction` 的方法引用：它会传递两个参数(键值对)并将第一个参数作为接收者，因此方法本身会像下面这样调用：

```
key.concat(value)
```

方法引用的第 1 个参数移到了接收者的位置，第 2 个参数(以及随后的参数，如果存在的话)会向左移动一个位置。因此，如下调用：

```
map.replaceAll(String::concat)
```

的结果是：

```
{alpha=alphaX, bravo=bravoY, charlie=charlieZ}
```

2.6.3 构造器引用

方法引用是对现有方法的句柄，与之类似，构造器引用是对现有构造器的句柄。构造器引用的创建语法类似于方法引用，只不过使用关键字 `new` 替换方法名。例如：

```
ArrayList::new  
File::new
```

与方法引用一样，对于重载构造器的选择是通过上下文的目標类型实现的。例如，在如下代码中，`map` 参数的目标类型是类型为 `String -> File` 的函数；为了与之匹配，编译器会选择带有单个 `String` 参数的 `File` 构造器。

```
Stream<String> stringStream = Stream.of("a.txt", "b.txt",  
    "c.txt");  
Stream<File> fileStream = stringStream.map(File::new);
```

2.7 类型检查

本节将会介绍 `lambda` 与某个函数类型匹配所需的条件，下一节将会介绍类型推断是如何在多个重载方法中找到与调用“最为匹配”的那个方法的。这两节内容都很详尽，你可以考虑在第一次阅读时将其跳过或是快速浏览一遍。

在本章前面我们看到如果 `lambda` 表达式所处的上下文提供了一个目标类型，那么我们就可以说该 `lambda` 表达式拥有单个类型。诸如此类的表达式(类型在一定程度上是由上下文决定的)称为聚合表达式(`poly expressions`)。方法与构造器引用也是聚合表达式；如果缺少上下文，那么表达式 `String::valueOf` 就可以指向名字为 `valueOf` 的 9 种不同的 `String` 方法。本节与下节将会探究

lambda 与方法及构造器引用类型检查的过程。不过，由于类型检查针对的是函数式接口的函数类型，因此首先需要对函数类型进行精确定义。

2.7.1 何为函数类型

在本章的后续部分，函数类型的概念对于理解类型检查的工作方式来说是非常重要的，因此我们需要好好地理解它。事实上，2.4 节给出的简单定义在大多数情况下都足够了：函数式接口的函数类型就是其唯一一个抽象方法的类型，也就是其类型参数，再加上参数类型、返回类型与抛出异常类型。

不过，有两个因素会导致这个简单定义变得复杂：首先，所有接口都声明了(如果没有显式声明，那就会隐式声明)与 `Object` 中的公共方法相对应的抽象方法，函数式接口的单个抽象方法并不在其中。很多时候，`Object` 方法都是隐式声明的，但并非总是这样；例如，`Comparator` 就显式声明了 `equals` 方法，它与 `Object` 的 `equals` 方法相匹配。在这种情况下，显式声明不会产生任何影响；`Comparator` 依然要满足函数式接口的定义。

第 2 个复杂因素就是两个接口可能包含不同的方法，但由于类型擦除的原因导致这两个方法出现了关联。例如，如下两个接口的方法：

```
interface Foo1 { void bar(List<String>arg);}  
interface Foo2 { void bar(List arg);}
```

就是重写相关的。如果一个接口的父接口包含了重写相关的方法，那么该接口的函数类型就是可以有效重写所有继承下来的抽象方法的单个方法的类型。在该例中，如果执行如下代码：

```
interface Foo extends Foo1, Foo2 {}
```

那么 Foo 的函数类型就是如下方法类型：

```
public void bar(List arg);
```

这些问题对于单个抽象方法这种简单情况来说是没有影响的，不过在遇到像 Comparator 这样的特殊情况时，能够理解背后的原因还是非常有价值的。

2.7.2 匹配函数类型

针对目标类型提供的函数式接口类型对 lambda 进行类型检查要求 lambda 表达式兼容于接口的函数类型。例如，如下赋值：

```
UnaryOperator<Integer> b = x ->x.intValue();
```

可以编译通过，因为 lambda 表达式兼容于 UnaryOperator<Integer> 的函数类型。下面列出了保证兼容性的条件⁵：

参数数量：lambda 与函数类型要有相同数量的参数。

参数类型：如果 lambda 表达式的类型是显式定义的，那么其类型就要与函数类型的参数相匹配；如果 lambda 的类型是隐式定义的，那么对于返回类型检查来说(下面会介绍)，其参数类型会被认为与函数类型的相同。

返回类型：

- 如果函数类型返回 void，那么 lambda 体必须是一个语句表达式(也就是说，表达式可以用作语句，就像方法调用或赋值一样)，例如：

```
(int i) -> i++;
```

⁵ 本节实际上是 *The Java Language Specification*(<http://docs.oracle.com/javase/specs/jls/se8/html/>)Java 8 版中 15.27.3 节的简化版。

注意这里语句表达式的值被丢弃了；此外，还可以使用没有返回值语句的块体⁶，例如：

```
(Thread t) ->{ t.start(); }
```

- 如果函数类型具有非 `void` 的返回值，那么 `lambda` 体就必须返回一个与赋值兼容的值。例如，下面这个 `lambda` 体返回一个 `int` 值，它会被赋给 `UnaryOperator` 的函数类型的 `Integer` 结果。

```
UnaryOperator<Integer> b = x ->x.intValue();
```

对于方才给定的兼容性条件来说，我们还需要添加进一步的条件来确保 `lambda` 表达式能够编译通过：

抛出类型：`lambda` 表达式可以抛出检查的异常，前提是函数类型声明抛出了该异常或是其父类型异常。

最后一个条件可能会导致问题。例如，假设我们想要声明一个方法来集中处理 `File` 的无参数方法所实现的各种不同的 I/O 操作所造成的 `IOException`。首先要声明：

```
<U> U executeFileOp(File f, Function<File,U>fileOp) { ... }
```

不过 `Function` 的函数类型并没有声明任何异常，因此下面这种调用：

```
executeFileOp(f, File::delete);
```

无法编译通过。相反，我们需要一个自定义的函数式接口：

```
@FunctionalInterface
interface IOFunction<T,R> {
```

⁶ 后面会经常使用“拥有值”或“拥有流”这两个术语来描述会返回值或流的方法。

```
R apply(T t) throws IOException;
}
```

这样，我们就可以根据意愿声明 `executeFileOp` 了：

```
<U> U executeFileOp(File f, IOFunction<File,U>fileOp) {
    try {
        return fileOp.apply(f);
    } catch (IOException e) {
        // centralized exception handling
    }
}
```

总结一下：**lambda** 可以像其他方法一样以两种相同的方式处理抛给它的异常——它可以隐式将其声明给调用者，也可以在 `catch` 块中处理。由于库函数式接口都没有声明任何异常，因此如果实现了用户定义的函数式接口，那么 **lambda** 只会将异常传递给调用者，就像示例中演示的那样。与之相反，我们将会在第 5.3 节中看到，对于流的处理来说，我们需要在 **lambda** 中采用迂回的方式处理检查的异常，因为流处理操作的参数都是库函数式接口。简而言之：在 Java 8 **lambda** 中，检查的异常的处理是一个问题。

2.8 重载解析

对于 2.7.2 节的兼容性标准来说，当函数式接口类型可以明确下来时，**lambda** 表达式的目标类型就非常清楚了。2.5 节列出的大多数目标类型上下文都提供了明确的上下文。但遗憾的是，当方法拥有不同的重载变体时，**lambda** 表达式最常用的上下文(方法调用站)也是最容易出现问题的。本节介绍的难题并不常见；但当其出现时却会导致令人沮丧的编译问题。如果你在设计 API，而

用户依靠你来避免这个问题，那就得好好研究这个话题了——凡事预则立！

下面对重载解析所面临的挑战做一个小结：调用不同参数个数的重载方法通常很容易区分，不过在参数个数相同的两个重载方法之间选择时则要求编译器知道所提供的参数类型。对于无类型的 `lambda` 来说，这就是个问题，因为在没有目标类型的情况下是无法推断出其类型的，而目标类型是由方法声明提供的，因此只有在完成重载解析后才能知晓这一切！我们可以通过施加一些规则来打破这个循环，要么就是让类型推断过程违反直觉和不可预测，要么就是强制用户提供 `lambda` 参数类型。为了避免这些不合需要的行为，设计者最终决定让重载解析过程在不确定的情况下更加宽容一些并设计出新的 API，从而可以通过隐式类型的 `lambda` 高效地实现重载解析。

2.8.1 `lambda` 表达式的重载

在探究使用类型推断所会遇到的困难之前，我们先来看看在简单直接的情况下其工作方式是怎样的。第1章(1.4节)介绍了方法 `Comparator.comparing`，给定一个键抽取器，它会根据其创建一个 `Comparator`。这是对 `comparing` 的一个重载(为了可读性，这里略掉了大多数类型边界)：

```
public static <T,U extends Comparable<U>>  
    Comparator<T>comparing(Function<T,U>keyExtractor) ❶
```

假设我们想要根据长度对字符串进行排序。给定明确的目标类型，例如赋值语句，`comparing` 会接收一个无类型的 `lambda` 表达式，然后通过它创建一个比较器来实现目标：

```
Comparator<String> cs = Comparator.comparing(  
    s ->s.length());
```

❷

这个高度简化的程序解释了❷中的lambda类型是如何解析的。首先,编译器要选择一组comparing重载方法,并根据这些方法检查lambda表达式。由于另一个重载方法接收两个参数,因此这组重载方法只包含❶。既然已经选择好了重载方法,那么❶的返回类型与❷的目标类型的匹配就会让编译器推断出在该例中T就是String。现在,lambda表达式可以与Function<T, U>的函数类型相匹配了:

```
public U apply(T t)
```

既然传递给 lambda 的参数类型 T 已经确定为 String,那么String.length的返回类型 int(会被装箱为 Integer)就可以替代 U 了。现在,所有类型都是已知的,❶中绑定在 U 上的类型是 Integer(它确保 lambda 的结果实现了 Comparable)。一切都是一致的,调用也可以编译通过。

到目前为止一切都很好。但是假设 Comparator 声明了 comparing 的另一个重载:

```
// removed from the JDK  
public static <T>  
    Comparator<T> comparing(ToIntFunction<T>  
        keyExtractor)
```

❸

在 Java 8 开发期间,这种重载确实存在。如果再次对❷进行类型检查,我们就会看到将这种情况取消的原因所在。遵循之前相同的流程,编译器首先会尝试选择针对于类型检查的重载方法。规则是这个行为必须在无类型 lambda 的类型检查之前进行(某些无类型的 lambda 可以在方法重载解析前进行类型检查,不过这种

行为被放弃了，因为设计者发现这么做会导致整个流程变得更加不一致和混乱)。

假定使用这个规则，那么提供给方法重载解析的信息就会少之又少。编译器无法使用赋值上下文所提供的目标类型，因为另一个重要规则要求无论上下文是什么，带有给定参数的方法调用必须被解析为相同的重载方法。它可以对方法调用使用其他参数，不过在该例中是没有参数的。它可以使用来自于 lambda 表达式的信息，不过对于兼容性条件来说，像这样的无类型 lambda 只能提供所接受的参数个数以及是否返回值等信息。在该例中，这些信息起不到任何帮助作用：lambda 接受一个参数并返回一个值，并且与两个候选的函数式接口的函数类型相匹配。这样，整个流程就卡住了，编译器会报错，给出的消息是“reference to comparing is ambiguous”。

那么我们该如何避免这种情况发生呢？如果 lambda 的参数类型是已知的，那么编译器就可以根据该信息，使用本节一开始介绍的算法来检查两个候选的函数类型。由于参数类型是 String，因此编译器会发现它们都与 lambda 兼容。接下来，编译器根据哪一个会返回最具体的结果来从这两个候选者当中进行选择：ToIntFunction 返回一个 int，Function<String, Integer> 返回一个 Integer，前者与 String.length 的返回类型更为接近。因此，下面的调用：

```
Comparator<String> cs = Comparator.comparing((String s)
->s.length());
```

可以编译通过，并被解析为❸。对于 Comparator.comparing 来说，Java 8 的设计者们不希望强迫用户像这样提供显式的 lambda 类型，因此它们使用新方法 comparingInt、comparingLong

与 `comparingDouble` 来替换接受的 `ToIntFunction`、`ToLongFunction` 与 `ToDoubleFunction` 的重载方法。对于其他 API 来说，如果函数式接口的参数个数不同，或是向重载方法提供的参数无二义性，那就可以提供不同的重载方法。如果你在设计 API，那么用户就会感谢你选择了这些实现方式。

2.8.2 方法引用的重载

方法与构造器引用又引入了新的难题：如果它们指向的方法或构造器是泛型的或重载的，那就是不精确的，因为其语法与显式的 `lambda` 类型是不一样的，因此你不能指定想要使用的精确类型。注意这是一种相对来说不太常见的情况：实际上，大多数方法既不是重载的，也不是泛型的。本节最后将会看到，可以通过使用类型化的 `lambda` 避免不精确的方法引用所带来的问题；此外，这也不是该问题的唯一解决方案。

举一个不精确的构造器引用的示例，`Exception::new` 可以指向如下两个 `Exception` 构造器：

```
public Exception()
public Exception(String message)
```

第 1 个匹配 `Supplier<Exception>` 的函数类型，第 2 个匹配 `Function<String, Exception>` 的函数类型。如果声明如下方法重载：

```
<T> void foo(Supplier<T> factory)
<T,U> void foo(Function<T,U> transformer)
```

那么调用：

```
foo(Exception::new);
```

将会出现编译错误，错误消息是 “reference to foo is

ambiguous”。不过，虽然我们无法像显式指定 lambda 类型那样让构造器引用更加精确，但还是有解决办法的：由于 foo 是泛型的，因此我们可以提供类型见证者来指定想要实例化的类型。该语法要求显式指定接收者：

```
this.<Exception>foo(Exception::new);
this.<String,Exception>foo(Exception::new);
```

当然，这么做只能解决具有不同数量的类型参数的泛型方法。假设我们想要调用如下两个重载方法之一：

```
void bar(IntFunction<String> f)
void bar(DoubleFunction<String> f)
```

在调用时传递指向String.valueOf的引用，这是一个静态方法，其重载版本分别匹配IntFunction<String>和DoubleFunction<String>。这样，只需要进行强制类型转换即可解决问题：

```
bar((IntFunction<String>) String::valueOf);
bar((DoubleFunction<String>) String::valueOf);
```

String::valueOf 的重载结果是其不同的函数类型匹配单个参数的可能类型，而后者用于区分 bar 的不同重载版本，因此问题就出现了。虽然这种情况可能会出现，但并不常见。如果真的出现了，那么请记住你总是可以使用相应的 lambda 表达式：

```
bar((double i) ->String.valueOf(i));
bar((int i) ->String.valueOf(i));
```

总结一下：如果类型推断或是重载解析失败了，那么你就需要提供更多的类型信息。有多种方式可以做到这一点，侵入性最小的方式是找到相应的类型化 lambda 表达式或是精确的方法引用。如果不行，那就提供一个类型见证者。从代码风格的角度来

看，强制类型转换应该是不得已而为之的手段，能不用尽量不用。

2.9 小结

到目前为止，我们已经介绍了为引入 `lambda` 需要做的所有语法变更；Java 8 的另一个主要变化是引入了默认方法，通过允许接口定义行为来支持 API 的演化。第 7 章将会介绍默认方法。本章介绍的一些细节看起来可能很复杂，不过总的来说，背后的想法是非常简单的，至少要比最终版本形成之前的中间阶段简单不少。

语言的演进总是困难的，因为每个新特性都会与现有特性存在或多或少的交互；复杂性就是个典型，这在本章中已经介绍过了，装箱/拆箱与无类型的聚合表达式对方法重载解析引入了很多复杂性。另外就是编写可以抛出检查的异常的 `lambda` 表达式是很困难的。在 `lambda` 语法的设计与实现中曾考虑过很多折中方案，其中与类型推断相关的方案最为引人注目，主要是因为它有时不如你想象的那么强大。这是因为在设计类型推断规则时最重要的推动力就是保持其简单性和一致性，这个想法也指导着整个语言的设计。